Abstraction Mechanisms for Virtual Avatars

Master's Thesis

submitted by

Nidanur Günay

at the

Department of Computer and Information Science

1. Reviewer: Prof. Dr. Oliver Deussen
2. Reviewer: Prof. Dr. Tiare Feuchtner

Konstanz, 2026

Abstract

Virtual reality applications increasingly rely on avatar representations to facilitate social presence and user embodiment. However, photorealistic avatar rendering can trigger the uncanny valley effect, where near-realistic human representations evoke discomfort and reduced trust rather than engagement. Non-photorealistic rendering (NPR) offers a principled alternative, deliberately abstracting avatar appearance while preserving expressiveness and identity.

This thesis presents the design, implementation, and evaluation of a real-time NPR stylisation system for virtual avatars. Eight techniques were developed iteratively: geometry-based inverted hull outlines, screen-space normal edge detection, Sobel edge detection with adaptive skin-to-clothing thresholding, Gaussian pre-filtered Sobel, hierarchical multi-cue edge detection combining depth, normal, and colour discontinuities, isotropic Kuwahara painterly filtering, and two composite techniques combining painterly abstraction with edge rendering. The techniques were implemented and compared across three avatar platforms — a Mixamo humanoid model, the Meta Avatars SDK, and Avaturn — to assess their visual behaviour and robustness under different mesh topologies and rendering pipelines. Avaturn was selected as the primary platform for the user study due to its facial rigging capabilities and higher photorealistic fidelity.

The user study investigates how visual abstraction affects the perceived trustworthiness of an avatar delivering advice. Participants watched short video scenarios in which a speaker — presented either as a real person, a photorealistic Avaturn avatar, or an NPR-stylised avatar — recommended one of two options in a practical decision situation. After each scenario, participants rated how trustworthy they found the speaker's recommendation. The results provide empirical insight into the relationship between avatar abstraction level and social trust, offering practical guidance for avatar design in applications where perceived credibility matters.

Keywords: non-photorealistic rendering, virtual reality, avatar abstraction, trust perception, edge detection, Avaturn, uncanny valley, Kuwahara filter.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Prof. Dr. Oliver Deussen, for his guidance, continuous support, and invaluable feedback throughout this thesis. His expertise in computer graphics and visual computing shaped the direction of this work in meaningful ways.

I am also deeply grateful to my co-supervisor, Prof. Dr. Tiare Feuchtner, for her insightful discussions, encouragement, and thoughtful advice — particularly on the human factors and user study aspects of this research. Her perspective was essential in connecting the technical work to its broader perceptual context.

Special thanks go to my colleagues at the Visual Computing Group at the University of Konstanz for the stimulating academic environment and helpful conversations throughout this journey.

Finally, I am grateful to my family and friends for their patience, understanding, and unwavering support.

Contents

List of Figures

List of Tables

Abbreviations

ANOVA	Analysis of Variance
API	Application Programming Interface
AR	Augmented Reality
CG	Computer-Generated
ESF	Extensible Shader Framework
FPS	Frames Per Second
GPU	Graphics Processing Unit
HCI	Human-Computer Interaction
HLSL	High-Level Shading Language
HMD	Head-Mounted Display
HSV	Hue, Saturation, Value (colour space)
LOD	Level of Detail
NPR	Non-Photorealistic Rendering
PBR	Physically-Based Rendering
RGB	Red, Green, Blue
SDK	Software Development Kit
SSS	Subsurface Scattering
UI	User Interface
URP	Universal Render Pipeline
UV	Texture Coordinate Axes (U and V)
VR	Virtual Reality

1 Introduction

1.1 Motivation

Communication through avatars has become a defining feature of virtual reality, extending well beyond its origins as a navigational convenience to serve as the primary medium through which users express identity, signal intent, and form social relationships in shared digital environments. The perceptual consequences of this shift are well documented. Avatar appearance exerts a measurable influence on social presence and interpersonal trust, meaning that the visual design of an avatar functions as a communicative act in its own right, shaping how users are perceived and how they engage with one another [1], [2].

Recent advances in generative AI have made photorealistic avatar creation more accessible than ever. Platforms such as Avaturn can now produce a fully rigged, expressively animated CG avatar from a single uploaded photograph, reconstructing facial geometry and surface appearance with a degree of fidelity that was difficult to achieve outside of dedicated production pipelines only a few years ago. The implicit promise of this technology is that a digital representation close enough to the real person will be received as one. Yet this is precisely where a well-known perceptual problem arises. Mori's uncanny valley hypothesis, first proposed in 1970, predicts that as a human representation approaches but falls just short of true photorealistic fidelity, it tends to evoke unease rather than familiarity [3]. An avatar that comes close to realism without quite reaching it can therefore undermine the very trust it was designed to support, a finding corroborated across robotics, computer graphics, and virtual environment research.

Non-photorealistic rendering (NPR) offers a different path through this problem. Rather than simulating accurate lighting and surface detail, NPR techniques deliberately stylise the rendered output to achieve artistic or communicative goals [4]. Cel shading, edge line drawing, and painterly filtering can each abstract an avatar's appearance in ways that sidestep the uncanny valley, while preserving enough expressiveness and readable identity that the character remains engaging. The open question is not whether this looks appealing. It is whether a deliberately abstracted avatar can still be trusted by the people interacting with it, or whether reducing realism introduces its own perceptual costs. That question has received surprisingly little attention in the context of modern, SDK-managed avatar systems.

1.2 Problem Statement

Applying NPR stylisation to photorealistic avatar systems is not straightforward. Production avatar platforms each manage rendering through their own shader architectures, material systems, and mesh topologies, and integrating custom visual effects into these

pipelines without disrupting avatar fidelity or performance requires working within constraints that most prior NPR research has not had to address. Existing work on NPR for virtual characters largely operates on custom assets built under full authorial control. How NPR techniques behave when applied across platforms with different rendering architectures, and which constraints prove most limiting in practice, is not well established.

Beyond this technical question, there is an open empirical one. Photorealistic avatar generation has become increasingly accessible, yet the uncanny valley problem suggests that realism alone does not guarantee a positive reception. Whether NPR abstraction can improve how an avatar is perceived, specifically in terms of trustworthiness, and how much stylisation is too much before credibility is lost, are questions that remain largely unanswered. They matter for any application where avatar credibility is at stake: social VR, remote consultation, education, and more.

This thesis addresses both challenges. It examines how NPR techniques can be applied across avatar platforms with differing rendering constraints, and asks whether the resulting visual abstraction affects the trust that viewers place in an avatar delivering a recommendation.

1.3 Contributions

This thesis makes the following contributions:

- **Multi-platform NPR shader system.** A real-time NPR stylisation system is developed and applied across three avatar platforms: a Mixamo humanoid model used for initial technique development, the Meta Avatars SDK integrated via the app-specific post-lighting fragment hook (`AppSpecificPostManipulation`), and Avaturn, which serves as the primary platform for the user study. The cross-platform implementation reveals the constraints and trade-offs specific to each rendering pipeline and demonstrates that the core NPR techniques generalise across different mesh topologies and material systems.
- **Iteratively developed stylisation techniques.** A series of techniques are designed, implemented, and compared through an iterative development process: toon shading with a quantized diffuse model, X-Toon extended toon shading with a depth-indexed 2D ramp, screen-space normal edge detection, Sobel edge detection with adaptive skin-to-clothing thresholding, Gaussian pre-filtered Sobel, hierarchical multi-cue edge detection combining depth, normal, and colour discontinuities, isotropic Kuwahara painterly filtering, Kuwahara combined with Sobel edge lines, and a composite technique combining Kuwahara, Gaussian Sobel, and hierarchical detection. Each technique addresses limitations observed in its predecessor, and all are selectable at runtime without recompilation. A final technique is selected for the user study based on visual assessment across all three avatar platforms.
- **In-VR parameter tuning interface.** A fully procedural, WorldSpace UI panel is implemented in Unity, enabling live exploration of all shader parameters from

inside the headset using controller raycasting. This tool was central to the iterative development process and can be reused for future shader experimentation.

- **Video-based user study on avatar abstraction and trust.** A user study is conducted using pre-recorded video scenarios in which a speaker, shown as either a real person, a photorealistic Avaturn avatar, or an NPR-stylised avatar, recommends one of two options in a practical decision situation. The video format is chosen deliberately: it is the only medium that allows a real human speaker and avatar counterparts to be compared under identical viewing conditions. The study provides empirical insight into how visual abstraction affects the perceived trustworthiness of an avatar's advice.

1.4 Thesis Outline

The remainder of this thesis is structured as follows:

Chapter 2 surveys prior work on NPR for 3D characters, avatar rendering and realism in VR, and trust and perception in virtual environments, positioning this thesis in the existing research landscape.

Chapter 3 introduces the technical and theoretical foundations required for this work, covering edge detection mathematics, the Kuwahara filter, the Meta Avatars SDK rendering pipeline, and concepts of trust and abstraction from human-computer interaction.

Chapter 4 presents the research questions, the overall system design, the iterative development strategy, and the user study design.

Chapter 5 describes the full implementation of the NPR shader system, covering all eight techniques in detail, the runtime tooling, and the rationale for the final technique selection used in the user study.

Chapter 6 reports the technical evaluation of each stylisation technique and presents the results of the user study, followed by a discussion of the findings and their implications.

Chapter 7 summarises the contributions of this thesis, reflects on its limitations, and outlines directions for future work.

2 Related Work

2.1 Non-Photorealistic Rendering for 3D Characters

Non-photorealistic rendering has a substantial history in computer graphics, though most of it predates the era of closed, SDK-managed avatar systems. Understanding this body of work requires distinguishing between the underlying techniques — edge detection, shading stylisation, and painterly filtering — and their specific application to animated human figures.

The foundational contribution to NPR lighting is the cool-to-warm shading model introduced by Gooch et al. [4], which replaced the continuous Lambertian diffuse term with a hue shift from cool shadows to warm highlights, mimicking the conventions of technical illustration. While the model targets engineering drawings rather than characters, its core insight — that shading can communicate form without physically simulating light — became a reference point for subsequent stylised rendering work. In the context of this thesis, the more direct technical heritage lies in outline extraction and edge-based effects rather than shading models.

Silhouette rendering has been approached from two geometrically distinct directions. Geometry-based methods generate outlines using the GPU rasteriser itself. The inverted hull technique, described by Lake et al. [5], expands back-facing polygons along vertex normals by a small offset, creating a slightly enlarged shell of the mesh. When the front-facing geometry is rendered on top, only the protruding rim remains visible as an outline. The approach requires no image-space computation, scales predictably with polygon count, and remains consistent regardless of camera distance — properties that make it well suited to animated characters in real-time applications. Isenberg et al. [6] provide a systematic survey of silhouette algorithms and conclude that geometry-based methods are robust and distance-independent, but structurally incapable of detecting internal surface detail. This limitation matters for avatar rendering, where facial features, clothing folds, and skin creases all constitute expressive visual cues that lie within the silhouette boundary.

Image-space methods operate on the rasterised output rather than on the mesh directly. Screen-space normal derivatives, computed via the hardware-accelerated `ddx`/`ddy` intrinsics available on all modern GPUs, detect abrupt changes in surface orientation without any additional draw calls or intermediate render targets. This approach captures surface creases and mesh boundaries that geometry expansion misses. The fundamental limitation is that edge thickness is measured in screen pixels rather than world-space units, so edges scale with camera distance. In an immersive VR context where the user moves relative to other participants, this produces edges that appear overly thick at close range and invisible at distance.

Texture-based edge detection targets a different layer of information. The Sobel operator [7], a discrete gradient approximation applied to the diffuse texture via a 3×3

convolution kernel, identifies luminance boundaries that correspond to clothing patterns, facial contours, and painted surface details. Unlike geometry-based methods, it responds to what was painted onto the surface rather than to the surface shape itself. The approach is well established in image processing [8], but differentiation inherently amplifies high-frequency noise. Marr and Hildreth [9] demonstrated that low-pass pre-filtering is a prerequisite for stable gradient computation, an observation that motivates the Gaussian pre-filtered variant explored later in this work.

Extended toon shading approaches have explored richer parameter spaces. Barla et al. [10] introduced X-Toon, which parameterises the shading tone ramp by both view angle and depth, enabling a two-dimensional lookup that encodes artistic intent unavailable in a one-dimensional quantisation step. On the painterly side, the Kuwahara filter — originally a noise-reduction tool in medical imaging — has been applied to NPR as a means of producing region-based painterly abstraction without explicit edge detection. Kyprianidis et al. [11] extended the isotropic variant with an anisotropic formulation aligned to local image structure, producing brush-stroke effects oriented along salient contours. The anisotropic extension requires a structure tensor computed from a prior render pass and cannot be implemented within a single-pass fragment hook — a constraint directly relevant to the integration target in this thesis.

Hatching represents a separate branch of character NPR in which tonal regions are conveyed by oriented stroke textures rather than flat fills or edge lines [12]. While visually distinctive, the approach depends heavily on stroke coherence across animation frames, a problem that has not been fully solved in real-time settings. It was not pursued in this work.

What connects nearly all of these contributions is an implicit assumption: the developer controls the rendering pipeline. Techniques are designed and evaluated against custom meshes, open rendering frameworks, or research engines where shader code, render passes, and material systems can be freely modified. This assumption does not hold for production avatar SDKs.

2.2 Avatar Rendering and Appearance in VR

Commercial avatar platforms have matured considerably beyond the research systems used in early social VR studies. The Meta Avatars SDK, used in Horizon Worlds and accessible to third-party developers, provides production-quality physically-based rendering with subsurface scattering for skin, procedural eye glints, hair rendering, and compute-based GPU skinning driven by body tracking data. The pipeline is generated and managed by Meta's Extensible Shader Framework (ESF), which exposes a small number of application-specific fragment hooks while keeping the core lighting computations proprietary and unmodifiable. This architecture maintains pipeline integrity across SDK updates and ensures consistent quality across hardware, but it substantially constrains where custom rendering logic can be inserted.

Prior NPR work applied to avatar systems has generally assumed open pipeline access. Studies applying toon shading or stylisation to VR avatars have used custom character

models with fully accessible shader code, or have modified appearance at the asset level before the render pipeline is involved. Applying NPR to a closed, compute-skinned, LOD-managed avatar at runtime — without SDK modification, without additional render passes, and without breaking the material management system — has not been addressed in the published literature to the author's knowledge.

The most directly related work on avatar stylisation and perception is Canales et al. [13], who conducted a controlled study of how avatar stylisation level affects perceived trust. Participants interacted with avatars at varying degrees of abstraction, from photorealistic to explicitly cartoon-like, and rated trustworthiness using social cognition instruments. The study found that moderate stylisation did not reduce trust relative to photorealistic rendering, and could in some conditions improve it, consistent with the uncanny valley framework. However, the stylisation was applied to custom assets in an open pipeline, and the technique variations compared were coarse-grained — the distinction between different edge detection approaches, painterly filtering methods, or composite techniques was not investigated. The work demonstrates that abstraction matters; it leaves open the question of which specific technical approach is most effective and whether these findings generalise to SDK-managed avatars.

Weidner and Broll [2] examined how avatar visual fidelity interacts with self-embodiment and social perception in video conferencing and VR contexts. Their findings indicate that the visual design of an avatar does not operate independently of the observer's sense of identification with it: fidelity effects on social judgements are mediated by embodiment. This has methodological implications for user studies that compare avatar conditions — a participant's prior exposure to and familiarity with a particular avatar style can influence their trust ratings independently of the stylisation itself.

2.3 Trust, Social Presence, and the Uncanny Valley

The uncanny valley hypothesis, first articulated by Mori [3], proposes that human affinity for a human-like representation increases with resemblance up to a point, then drops sharply before recovering near perfect replication. The drop is attributed to a mismatch: a near-realistic representation triggers expectations calibrated for real humans, but subtle violations — in motion timing, skin texture, eye movement — produce unease rather than familiarity. MacDorman and Ishiguro [14] provided empirical support for this in the context of humanoid robots, documenting the eeriness ratings that emerge when a representation is close but not convincingly human.

Modern avatar systems sit squarely in the region of this curve that is most problematic. They achieve surface realism sufficient to trigger high perceptual expectations, but cannot fully meet those expectations in motion timing, facial micro-expressions, or gaze behaviour. The practical consequence for social VR is that a photorealistic avatar may actively undermine the sense of trustworthiness that it was designed to support. NPR stylisation provides a principled alternative: by moving the representation away from near-photorealism and into a clearly artistic register — cel shading, ink-line treatment,

painterly filtering — the avatar exits the uncanny valley zone rather than attempting to cross it.

The relationship between avatar appearance and social presence has been studied extensively. Nowak and Biocca [1] demonstrated that anthropomorphism and perceived agency are distinct constructs that contribute independently to social presence and copresence. Counterintuitively, very high levels of visual anthropomorphism did not consistently produce higher social presence scores. This suggests that the social and perceptual response to an avatar is not a simple monotonic function of visual realism, and that stylisation does not necessarily carry a social cost.

Trust in the context of virtual agents is related to social presence but distinct from it. A user can experience strong co-presence with an avatar while trusting the entity it represents very little, or trust a stylised character precisely because it does not trigger uncanny valley discomfort. The literature on avatar trust has largely compared coarse conditions — human versus clearly robotic, photorealistic versus cartoonish — without exploring the design space within NPR itself. Whether a Kuwahara-painterly avatar is trusted differently from a Sobel-edge-line avatar, or whether hierarchical multi-cue edge detection produces a better trust outcome than a simpler screen-space method, are questions that have not been addressed.

2.4 Summary and Positioning

The technical NPR literature provides a rich set of established techniques, developed and validated in open rendering environments. The social VR literature demonstrates that avatar appearance significantly affects trust and social presence, and that the uncanny valley poses a genuine design challenge for photorealistic avatar systems. Canales et al. showed that stylisation can improve trust outcomes; Nowak and Biocca established that the appearance–presence relationship is non-linear. What is missing is the intersection: a systematic investigation of fine-grained NPR technique choice on a production, SDK-managed avatar system, paired with an empirical measure of the trust response those choices produce.

This thesis addresses that gap. It integrates a suite of NPR techniques into the Meta Avatars SDK under real production constraints, evaluates the visual and performance trade-offs across techniques, and measures the trust implications of different abstraction levels in a user study.

3 Background

This chapter introduces the technical and theoretical foundations required for the work described in subsequent chapters. It covers the concept of virtual avatars and visual representation, edge detection mathematics, the Kuwahara painterly filter, and the human factors concepts of trust and social presence relevant to the user study.

3.1 Mathematical Notation

Throughout this thesis, scalars are denoted by italic letters x, vectors by bold lowercase letters $\mathbf{x}$, and matrices by bold uppercase letters $\mathbf{X}$. The i-th element of vector $\mathbf{x}$ is written x_i. Convolution of a signal I with a kernel K is written $K * I$. In shader pseudocode, `ddx(f)` and `ddy(f)` refer to the hardware-computed screen-space partial derivatives of a value `f` with respect to the horizontal and vertical screen axes, respectively.

3.2 Edge Detection

An edge in an image is conventionally defined as a location where intensity changes rapidly. Detecting such locations involves estimating the spatial gradient of the image intensity function, then applying a threshold to identify pixels where the gradient magnitude exceeds some criterion.

3.2.1 Gradient-Based Detection

Given an image $I(x, y)$, the gradient is the vector $\nabla I = (\partial I/\partial x,\, \partial I/\partial y)$. Its magnitude

$$M = \|\nabla I\| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2} \tag{3.1}$$

provides a scalar measure of edge strength at each pixel. In practice, the partial derivatives are approximated by discrete convolution with finite-difference kernels.

The Sobel operator [7] estimates the gradient using two 3×3 kernels:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \qquad G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}. \tag{3.2}$$

G_x responds to horizontal intensity transitions (vertical edges); G_y responds to vertical transitions. The gradient magnitude is $M = \sqrt{G_x^2 + G_y^2}$, computed pointwise after convolving the image with each kernel separately. The ± 2 weights along the central row and column give the Sobel operator a small amount of spatial smoothing relative to the simpler Prewitt kernel, improving noise robustness marginally.

The Roberts Cross operator uses a 2×2 diagonal kernel pattern instead:

$$R_1 = \begin{bmatrix} +1 & 0 \\ 0 & -1 \end{bmatrix}, \qquad R_2 = \begin{bmatrix} 0 & +1 \\ -1 & 0 \end{bmatrix}. \tag{3.3}$$

The edge magnitude is $|R_1| + |R_2|$, which approximates the gradient diagonal without a centre sample. Roberts Cross is faster and more local than Sobel but more sensitive to noise. In this thesis it is used specifically for colour edge detection within the hierarchical technique, where its two-sample footprint keeps the per-fragment sample count low.

3.2.2 Screen-Space Normal Derivatives

Modern GPU pipelines expose per-fragment screen-space derivatives of any interpolated value via the `ddx` and `ddy` intrinsics, computed using the finite difference between adjacent 2×2 pixel quads. Applying this to the world-space surface normal $\mathbf{N}$ gives

$$M_N = \sqrt{\|\texttt{ddx}(\mathbf{N})\|^2 + \|\texttt{ddy}(\mathbf{N})\|^2}, \tag{3.4}$$

which responds to abrupt changes in surface orientation — creases, mesh boundaries, and regions of high curvature. This computation requires no additional texture samples and executes at negligible cost relative to the surrounding fragment shader. The significant practical limitation is that screen-space derivatives are measured in pixel units. A fixed-world-space edge — the crease between a jacket lapel and its collar, for instance — projects to a varying number of pixels depending on camera distance, causing edge thickness to scale with the viewpoint rather than with the surface.

3.2.3 Gaussian Pre-Filtering

Differentiation amplifies high-frequency components. Marr and Hildreth [9] established that stable edge detection requires low-pass smoothing before gradient computation. Because convolution and differentiation are both linear operations, the smoothed gradient can be written as

$$\nabla(G_\sigma * I) = (\nabla G_\sigma) * I, \tag{3.5}$$

where G_σ is a Gaussian kernel with standard deviation σ. Canny's influential edge detector [8] built on this principle: smooth with a Gaussian, compute the gradient of the smoothed image, and apply non-maximum suppression to obtain thin, stable edge maps. In a single-pass fragment shader, a true two-pass separable convolution is not available. The Gaussian can instead be approximated by sampling the source texture at a fixed set of offsets with precomputed weights, accepting that the smoothed values are recomputed independently at each sample position.

3.3 The Kuwahara Filter

The Kuwahara filter [11] is a structure-preserving smoothing operator that produces piecewise-uniform regions with sharp inter-region boundaries — a visual effect that

resembles painterly brushwork. It operates by dividing the neighbourhood around each pixel into k overlapping subregions, computing the mean and variance of pixel values within each subregion, and setting the output to the mean of whichever subregion has the lowest variance. Formally, for subregions $\{Q_i\}_{i=1}^{k}$ and their respective means μ_i and variances σ_i^2:

$$F_{\text{Kuwahara}}(x, y) = \mu_j, \qquad j = \arg\min_i \sigma_i^2. \tag{3.6}$$

The effect is that pixels on uniform surfaces are assigned the mean colour of their most locally uniform neighbourhood, while pixels near strong boundaries pick the subregion that falls entirely on one side. Edges are therefore sharpened rather than blurred, even as internal regions are smoothed toward a uniform painterly fill.

The classical isotropic formulation uses four quadrant-shaped subregions of equal size, producing a rotationally symmetric filter that does not prefer any orientation. An anisotropic extension by Kyprianidis et al. [11] aligns the filter kernel with local image structure by computing a structure tensor from a prior render pass, producing brush-stroke effects oriented along salient contours. This produces superior results visually, but requires an intermediate render target that is incompatible with a single-pass fragment hook.

3.4 Virtual Avatars and Visual Representation

In virtual environments, an avatar is a digital body that represents a user within a shared space. The term covers a wide range of forms, from abstract geometric markers to fully articulated, photorealistic human figures, and the choice of representation carries consequences that extend well beyond visual preference.

Avatar representations can be placed along a continuum of anthropomorphism and visual fidelity. At one end, abstract or stylised avatars retain the structural properties of a humanoid figure without committing to the surface detail of a real person. At the other, photorealistic avatars attempt to reproduce the user's physical appearance with enough accuracy to approach the visual fidelity of a photograph. Where an avatar sits along this continuum has measurable consequences for how users are perceived, how much social presence is felt in the interaction, and how the avatar is trusted [1], [15].

Avatar appearance also shapes the behaviour of the person embodying it. Yee and Bailenson [16] demonstrated through a series of experiments that users assigned more attractive or physically taller avatars adopted behaviours consistent with those characteristics: they disclosed more personal information, negotiated more assertively, and maintained shorter interpersonal distances. This tendency, termed the Proteus effect, illustrates that visual self-representation in virtual environments is not a passive interface element but an active influence on how users act and present themselves.

Contemporary avatar platforms have made high-quality avatar creation accessible without requiring manual artistic work. Services such as Avaturn reconstruct a personalised, fully rigged avatar from a single uploaded photograph using learned three-dimensional reconstruction, producing a mesh with facial blendshape animation support that integrates

directly into standard game engine pipelines. This accessibility makes individualized photorealistic representation practical for a wide range of applications, and it sharpens the perceptual stakes: an avatar that looks like a real person will inevitably be compared to one, and any shortfall in that comparison has consequences both for immersion and for how the user is received by others.

3.5 Trust and Social Presence in VR

Trust in the context of human-avatar interaction is typically understood as a social judgement — the degree to which an observer is willing to rely on the entity represented by the avatar. It is related to but distinct from social presence, which refers to the subjective sense of sharing a space with another being. Both constructs are influenced by avatar appearance, but through different mechanisms. Social presence depends strongly on the perceived anthropomorphism of the avatar and the responsiveness of its motion [1]. Trust is additionally sensitive to perceived competence, predictability, and familiarity — including the absence of uncanny valley discomfort [3].

In user studies, social presence is most commonly measured with self-report instruments including the Social Presence Questionnaire [1] and the items from Witmer and Singer's sense-of-presence measure. Trust has been assessed with scales drawn from social psychology adapted for HCI contexts, where items probe dimensions such as reliability, benevolence, and integrity. The Godspeed questionnaire [14] provides a validated set of semantic differential items for rating human-likeness, animacy, and eeriness, which have been used to characterise uncanny valley responses to robotic and avatar stimuli.

A consistent finding across this literature is that the relationship between visual fidelity and perceived trustworthiness is not linear. Near-photorealistic representations can produce lower trust scores than clearly stylised ones, consistent with the uncanny valley hypothesis. This non-linearity motivates the experimental design of this thesis, which compares a range of NPR abstraction levels rather than simply contrasting a stylised condition against the photorealistic baseline.

4 Methodology

This chapter describes the research questions guiding this work, the three avatar platforms used as development and evaluation targets, the nine NPR stylisation techniques developed across those platforms, and the design of the user study that evaluates their perceptual consequences.

4.1 Research Questions

Three research questions structure the contributions of this thesis.

RQ1: Can NPR stylisation techniques be implemented and applied across avatar platforms with different rendering architectures, specifically Mixamo, the Meta Avatars SDK, and Avaturn, and what technical constraints does each platform impose on achievable effects?

This question is primarily technical and addresses two related sub-problems. First, it asks whether a viable integration path exists for each platform, given that the three platforms differ substantially in mesh topology, shader accessibility, and rendering pipeline structure. Second, it asks what constraints each integration path places on the set of NPR effects that can be achieved, which determines which techniques from the full design space are realisable on each platform.

RQ2: Which NPR technique provides the best trade-off between visual quality, expressiveness, and rendering performance when applied across avatar types?

For this question, *visual quality* refers to the perceptual clarity and coherence of the stylised output; *expressiveness* refers to the degree to which facial features and body contours remain readable under stylisation; and *rendering performance* refers to the per-frame GPU cost measured as frame time. This question drives the iterative development of the nine techniques. Each technique operates on a different definition of a visual boundary, works in a different coordinate space, and incurs a different sample cost per fragment. Answering RQ2 requires both qualitative visual assessment of expressiveness and feature readability, and quantitative frame-time measurement, evaluated across all three avatar platforms. Where these two assessments conflict, expressiveness is given priority because the user study requires facial animation to remain legible.

RQ3: How does the level of visual abstraction affect the perceived trustworthiness of an avatar delivering a recommendation?

This is the central empirical question. The phrase *level of visual abstraction* refers here to the contrast between two discrete conditions: a photorealistic avatar with standard PBR shading and an NPR-stylised avatar rendered with the technique selected from RQ2. A continuous scale of abstraction would require many conditions; the study operationalises the question as a targeted comparison between these two points, with a real human

speaker providing the trust ceiling baseline. The formal hypotheses tested are stated in Section 4.5.1.

4.2 Avatar Platforms and Rendering Constraints

Three avatar platforms are used across the development and evaluation phases of this thesis. They were selected to cover a range of shader accessibility and integration constraints: Mixamo provides an unconstrained baseline where all URP rendering features are available; the Meta Avatars SDK represents the constrained end of the spectrum, where NPR logic must be injected through a restricted hook interface; and Avaturn occupies a middle position, providing full URP access combined with the photorealistic facial quality required for the user study.

Mixamo. The Mixamo platform provides a web-based library of rigged humanoid characters that export into Unity as standard FBX assets [17]. In this thesis, the Mixamo character Jody serves as the primary prototyping target during the initial development phase. Because Mixamo avatars are standard skinned meshes with no proprietary shader system, all NPR techniques can be applied without restriction directly in the Unity Universal Render Pipeline (URP). This makes Mixamo the least constrained of the three platforms and the appropriate starting point for technique exploration before more restricted platform integrations are addressed.

Figure 4.1: Jody (Mixamo): photorealistic baseline used for initial NPR technique development.

Meta Avatars SDK. The Meta Avatars SDK provides an avatar system built for production deployment on Meta Quest hardware, offering body tracking, level of detail (LOD) management, and a proprietary Extensible Shader Framework (ESF) [18]. The ESF generates platform-specific HLSL shader code from parameterised templates; the generated source is not intended to be modified directly. Custom rendering logic must instead be injected through hook functions exposed by the ESF at defined points in the pipeline. This restricts both what NPR effects can be achieved and how they must be implemented. A further constraint is the SDK's LOD management, which replaces avatar materials automatically

as camera distance changes; the approach used to maintain custom shader assignments across these transitions is described in Chapter 5.

Figure 4.2: Meta avatar: SDK-driven avatar used to test NPR integration within the Extensible Shader Framework.

Avaturn. Avaturn is an avatar creation service that uses AI to reconstruct a rigged, blendshape-animated avatar from a single uploaded photograph. Avaturn exports avatars in the `.glb` format, the binary container of the glTF 2.0 standard, which packages geometry, PBR materials, skeleton rig, and blendshape targets into a single file. Because Unity does not natively support glTF at runtime, the GLTFast package is used to load the asset at runtime [19]; the import pipeline is described in Chapter 5. Once imported, the avatar is a standard skinned mesh with PBR materials and no proprietary shader constraints, giving full access to the URP rendering pipeline. Avaturn was selected as the primary platform for the user study because its photorealistic facial fidelity provides a convincing baseline condition for the trust comparison, and its blendshape animation support enables the facial expression delivery required in the video scenarios.

Figure 4.3: Avaturn avatar: AI-reconstructed from a photograph, used as the primary platform for the user study.

Table 4.1: Summary comparison of the three avatar platforms across dimensions relevant to NPR integration.

	Mixamo (Jody)	**Meta Avatars SDK**	**Avaturn**
Shader access	Full URP, unrestricted	Hook injection only (ESF post-lighting)	Full URP, unrestricted
LOD management	Manual	SDK-controlled, reverts materials	Manual
Import pipeline	FBX, Unity standard	SDK runtime loader	glTF 2.0 via GLTFast
Transparency	Explicit alpha cutout	SDK-internal	Explicit alpha cutout
Primary role	Technique prototyping	Constraint validation	User study

4.3 NPR Technique Design

Nine NPR stylisation techniques were developed through an iterative process structured in three phases corresponding to the three avatar platforms. Each technique addresses at least one observed limitation of earlier approaches in the sequence: limitations in interior edge coverage motivated derivative-based detection; limitations in noise robustness motivated pre-filtering; and limitations in edge selectivity motivated multi-cue fusion and painterly abstraction. Development began on the Mixamo platform, where unrestricted shader access allowed techniques to be prototyped and iterated freely; it then continued within the Meta SDK environment, where hook constraints shaped several architectural decisions; and was finally extended to Avaturn to confirm that each technique transfers to the user-study platform without modification. What follows is a conceptual overview of each technique's motivation and visual behaviour across the three platforms. Full implementation details are provided in Chapter 5.

4.3.1 Shared Rendering Structure

All techniques share a common two-pass rendering structure. The first pass is a dedicated silhouette outline pass; the second is the main forward-lit pass in which the technique-specific NPR surface effect is computed. Separating these concerns into two passes allows the outline width, colour, and depth behaviour to be tuned independently of the surface shading, and ensures that the outline geometry is always drawn at a consistent depth regardless of the surface material's transparency mode.

Inverted Hull Outline

The silhouette outline is generated using the inverted hull method, a common hybrid technique for rendering outlines that involves the enlargement of back-facing polygons, as detailed by Isenberg et al. in their survey of silhouette algorithms for polygonal models [6]. The technique, also referred to as the shell method, exploits GPU face culling to produce outlines without requiring edge detection or additional render passes. It operates across two dedicated rendering passes:

1. **Outline Pass.** The mesh is rendered with front-face culling enabled, displacing each vertex outward along its world-space surface normal by a configurable distance w:

$$\mathbf{P}' = \mathbf{P} + \mathbf{N} \cdot w \tag{4.1}$$

   where $\mathbf{P}$ is the original vertex position and $\mathbf{N}$ is the world-space surface normal, both expressed in world coordinates. Only the back-facing geometry is rasterised, producing an enlarged shell around the avatar.

2. **Main Pass.** The mesh is subsequently rendered with standard back-face culling. The front-facing surfaces overwrite the interior of the previously rendered shell, leaving only the protruding rim visible as a silhouette border of controllable thickness.

Lake et al. establish that this approach is well-suited for real-time animation and NPR because it provides a simple and scalable solution that avoids the computational overhead of complex mesh pre-processing or CPU-side silhouette calculations [5]. Because the expansion occurs entirely on the GPU during the vertex transformation stage, the technique maintains predictable performance characteristics across complex topologies and meshes. A small constant depth bias is applied to shift the outline geometry slightly toward the camera in clip space, preventing z-fighting where the expanded shell and the original surface occupy similar depth values.

The inverted hull method has known limitations relevant to avatar rendering. Surfaces with degenerate or coincident normals, such as flat planes, produce no visible outline because vertex displacement along a zero-divergence normal field does not create a shell. Furthermore, the method cannot generate outlines at interior surface boundaries: creases between body regions, clothing seams, and facial contours produce no edge, because only

object silhouettes are captured [6]. This structural limitation is shared by all geometry-based outline methods and motivates the interior edge detection approaches introduced from V2 onward.

(a) Jody (Mixamo): inverted hull silhouette outline.

(b) Avaturn: inverted hull silhouette outline.

(c) Meta avatar: inverted hull outline pass enabled.

Figure 4.4: Inverted hull geometry outline applied to all three avatar platforms. The Meta Avatar screenshot (c) required a full application build and deployment to a physical headset, as the SDK's avatar system depends on live body-tracking input. The slight floor penetration visible at the avatar's feet in (c) reflects a tracking boundary offset during capture and is not a property of the outline technique.

Alpha Cutout for Transparency

Avatar geometry on the Mixamo and Avaturn platforms includes semitransparent elements: eyelashes and hair are represented as flat cards with alpha-masked textures, where the visible shape is defined by discarding fragments below an opacity threshold. Without explicit handling, the expanded hull would be drawn over the full rectangular extent of each card, producing solid rectangular borders rather than outlines that conform to the visible hair silhouette. To handle these correctly, alpha testing is applied using the GPU's built-in fragment clip operation to discard any fragment whose texture alpha value falls below a configurable cutoff threshold α_{cutoff}:

$$\text{if } \alpha_{\text{texture}} < \alpha_{\text{cutoff}} \quad \text{then discard} \tag{4.2}$$

Alpha testing is applied in both the outline pass and the forward-lit pass to maintain silhouette consistency. A threshold of $\alpha_{\text{cutoff}} = 0.1$ is used, chosen empirically as the lowest value that reliably excludes fully transparent card regions while preserving the visible hair silhouette without clipping semi-transparent anti-aliased edges. Unlike alpha blending, this binary approach requires no depth-sorting and adds negligible runtime cost, making it well-suited for real-time VR rendering [5].

The Meta Avatar platform handles transparency through a different mechanism and does not require explicit alpha testing in the outline pass. The SDK manages hair and eyelash geometry through a dedicated multi-pass material system in which each rendering pass is identified by a string tag (in Unity's shader model, these are referred to as *named passes*). The outline is implemented as one such named pass within the same multi-pass shader. The SDK's fragment pipeline processes hair and eyelash transparency internally before the outline hook is invoked, so transparent fragments are already discarded before the outline colour is applied. This means the shared two-pass structure is fully preserved on the Meta SDK platform, but the transparency handling mechanism differs from the explicit cutout used on the other two platforms. The full technical details are discussed in Section 5.3.2.

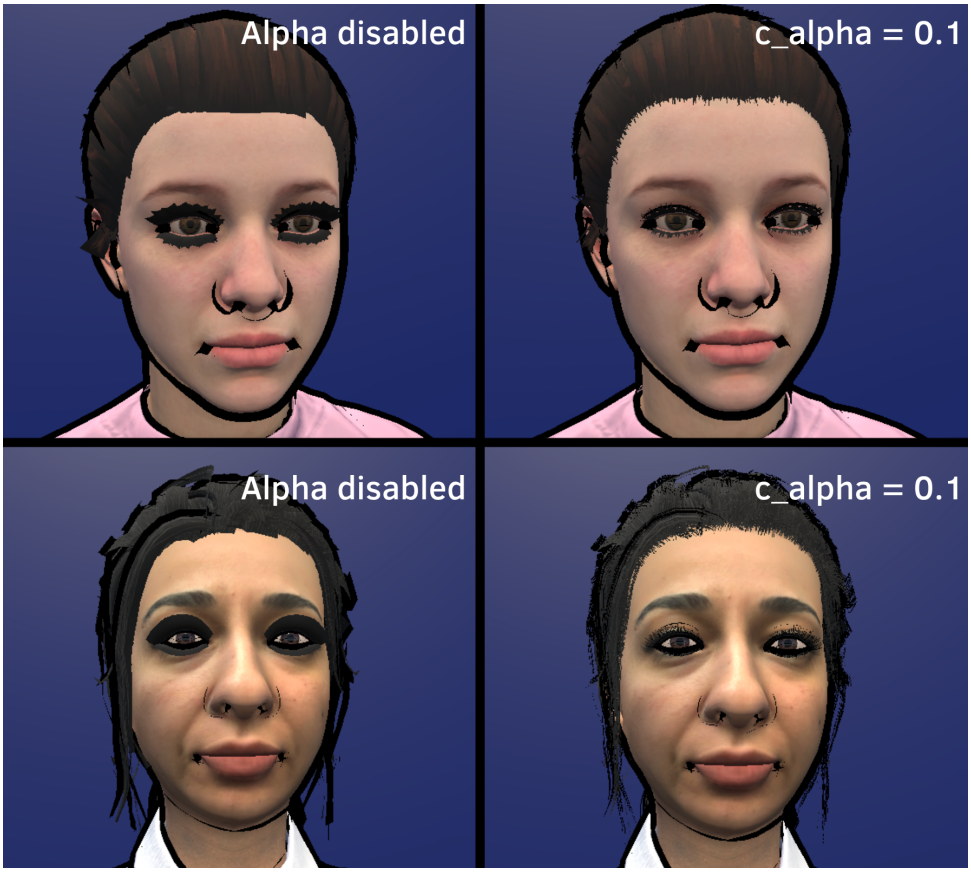


Figure 4.5: Effect of alpha cutout on the inverted hull outline pass for Jody (Mixamo, top row) and Avaturn (bottom row); outline width set to 0.005 on both avatars. Left column: alpha testing disabled; the outline is extruded over the full rectangular hair card mesh, producing visible borders around the eyelash and hair regions. Right column: alpha cutout enabled at $\alpha_{\text{cutoff}} = 0.1$; the hull is confined to the opaque silhouette.

4.3.2 V1 — Toon Shading

Toon shading, also known as cel shading, is chosen as the V1 baseline because it is the canonical NPR departure from photorealism: replacing the continuous Lambertian gradient with discrete, uniform colour bands eliminates the smooth shading that signals realistic rendering and produces the characteristic flat, illustrative appearance of hand-drawn animation [5]. Gooch et al. establish that non-photorealistic lighting models can

effectively translate three-dimensional volumes into distinct colour regions by departing from smooth Lambertian reflectance [4]. Lake et al. extend this principle to cel animation by substituting a quantised stepped function for the continuous lighting value, discretising it into uniform tonal bands, and demonstrate that this approach is practical on GPU hardware at interactive frame rates [5]. The shared inverted hull pass provides the outer silhouette; the forward-lit pass performs the technique-specific surface computation. The three avatar platforms diverge in how they access the lighting input, requiring distinct implementations of the quantisation stage.

On Mixamo and Avaturn, the Lambert diffuse term $\mathbf{N} \cdot \mathbf{L}$ is computed per fragment from the scene directional light and the interpolated surface normal. This continuous irradiance value is partitioned into n discrete shading bands using the floor quantisation:

$$t = \frac{\lfloor s \cdot n \rfloor}{n} \tag{4.3}$$

where s is the smoothed per-fragment lighting value and n is the configurable number of shading bands [5]. The floor operation truncates the continuous value into discrete intervals, producing the stepped tonal appearance of cel animation. Four bands are used as the design default, providing a highlight, mid-tone, shadow, and deep shadow: four perceptually distinct regions that convey three-dimensional form without continuous gradients, consistent with the tonal range demonstrated by Lake et al. for stylised real-time characters.

Band boundaries are temporally unstable under animation without softening: as the surface normal rotates past a quantisation threshold, the entire surface patch would snap instantaneously between tonal levels, creating a flickering artefact visible during movement. A smoothstep function is therefore applied at each boundary to convert this instantaneous transition into a short crossfade, preserving the stepped visual character while suppressing temporal aliasing. A shadow strength parameter α then modulates the depth of the banded effect by interpolating between the fully quantised value and uniform full brightness:

$$t' = \text{lerp}(1.0,\ t,\ \alpha) \tag{4.4}$$

When $\alpha = 0$ the surface appears uniformly lit regardless of orientation; when $\alpha = 1$ the full quantised result is applied. This provides independent control over shadow depth without altering the band count or boundary positions.

The final fragment luminance adds a constant ambient term and a view-dependent rim highlight. The ambient term prevents fully shadowed regions from appearing black, keeping the avatar readable against any background. The rim highlight uses a Fresnel-inspired approximation following Schlick [20]:

$$r = \left(1 - \text{saturate}(\mathbf{V} \cdot \mathbf{N})\right)^p \tag{4.5}$$

where $\mathbf{V}$ is the view direction, $\mathbf{N}$ the surface normal, and p a user-controlled exponent. Unlike Schlick's physically derived exponent of five, p is treated here as a stylistic parameter. The rim term is additive and brightens grazing-incidence fragments, visually

separating the avatar from the background at the silhouette edge even when the outer outline is narrow. The combined lighting output is:

$$L_{\text{total}} = L_{\text{light}} \cdot t' + L_{\text{ambient}} \tag{4.6}$$

Cast shadows from the scene shadow map are deliberately excluded in V1. Including shadow map contributions would introduce a second visual variable into the baseline comparison; shadows would be a confound that V1 is designed to avoid in order to isolate the toon quantisation effect itself.

On the Meta SDK platform the post-lighting hook receives a fully composited PBR colour that already incorporates subsurface scattering, anisotropic hair shading, and image-based lighting; the intermediate $\mathbf{N} \cdot \mathbf{L}$ term used in Equation 4.3 is not accessible at this pipeline stage. Recomputing irradiance from scratch would discard the physically accurate material response already computed by the SDK renderer. The quantisation therefore operates on the composited RGB output directly, rounding each channel to the nearest discrete step, which is a round-to-nearest posterisation rather than the floor-to-band operation of Equation 4.3. The two operations differ mathematically but achieve the same perceptual outcome: discrete, flat colour regions that replace smooth gradients. Because the SDK input already embeds shadow and ambient contributions, no auxiliary lighting terms are required.

Jody and Avaturn produce a clean graphic appearance with flat bands defined by surface-to-light angle, an ambient fill, and a rim highlight, while the Meta result retains the richer material response of its physical renderer within quantised tonal levels. The critical limitation shared by all three platform implementations is that quantisation operates entirely on surface-to-light angle or composited colour: no edge signal is generated at interior surface boundaries. Facial features, including eye sockets, the nose bridge contour, and lip boundaries, produce no visible line; clothing folds and painted texture boundaries are equally invisible, as Isenberg et al. note that geometry-based methods are inherently limited to object boundaries and cannot convey internal surface detail [6]. Because the user study measures trust responses to a speaking avatar, facial expression readability is a primary requirement: a technique that leaves the face as a featureless tonal field cannot adequately convey the contour changes that communicate engagement and sincerity. This absence of interior edge signal is the defining limitation of V1. V1.2 extends the tonal control of the shading model before interior edge detection is introduced in V2.

(a) Jody (Mixamo): four toon bands ($n = 4$), NdotL floor quantisation, additive ambient and rim highlight.

(b) Avaturn: identical NdotL quantisation; alpha-cutout outline confines the hull to the visible hair silhouette.

(c) Meta SDK: RGB posterisation on the composited PBR output; shadow and ambient precomposited by the SDK renderer.

Figure 4.6: V1 (Toon Shading) across all three avatar platforms, using four shading bands with smoothstep boundary transitions (Equations 4.3–4.6). Jody and Avaturn apply NdotL floor quantisation; the Meta SDK applies round-to-nearest posterisation on the fully composited PBR colour.

4.3.3 V1.2 — X-Toon Extended Toon Shading

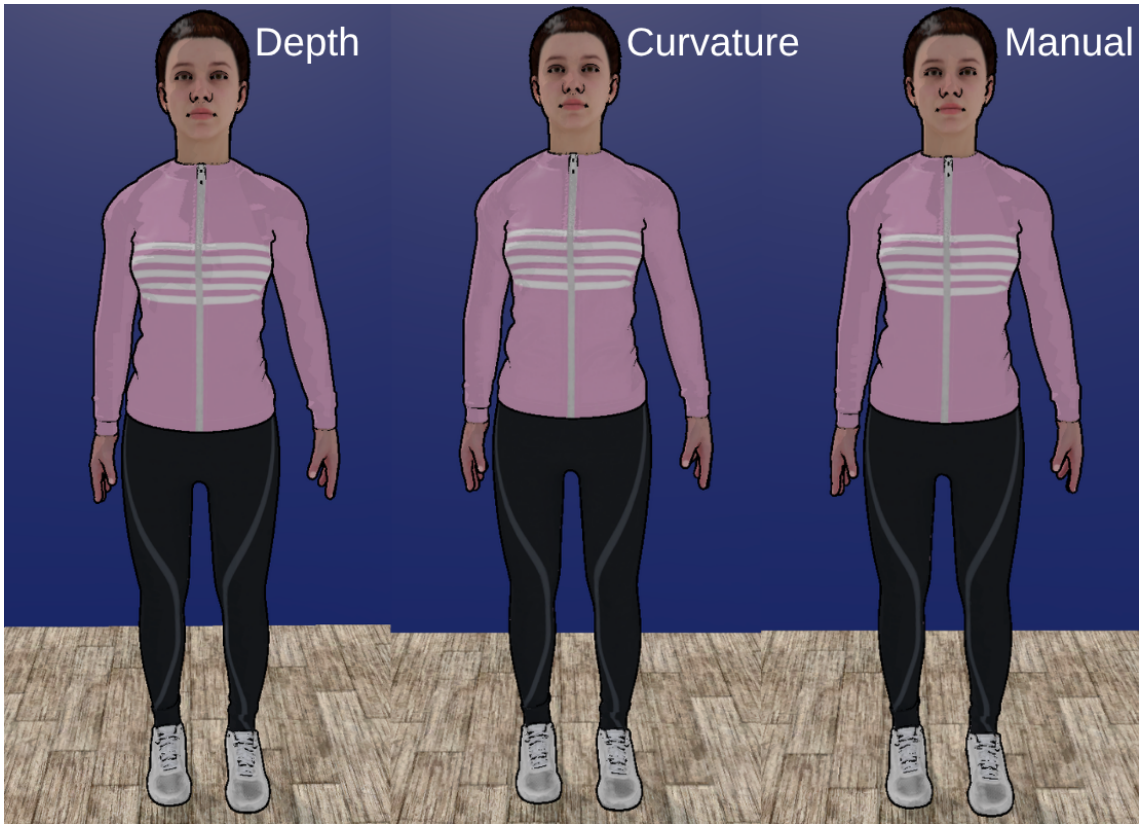
The second technique replaces the one-dimensional NdotL ramp of V1 with the two-dimensional ramp of Barla et al. [10]. The horizontal axis (U) encodes lighting intensity; the vertical axis (V) encodes an *abstraction level*: a per-fragment scalar that increases with camera distance in depth mode, with surface flatness in curvature mode, or is set to a constant in manual mode. The designer shapes the shading character by authoring the ramp texture rather than by adjusting shader parameters, giving direct visual control over tone and depth response.

As the abstraction level rises, the U coordinate is additionally compressed toward the ramp midpoint so that both lit and shadowed values flatten toward the same neutral tone, while the smoothstep transition width simultaneously widens to soften band edges. On Mixamo and Avaturn, where the full tangent frame is available, an optional abstract normal map can be blended with the interpolated geometry normal via TBN-space interpolation; Barla et al. term this technique Normal Field Abstraction [10], which converges normals within

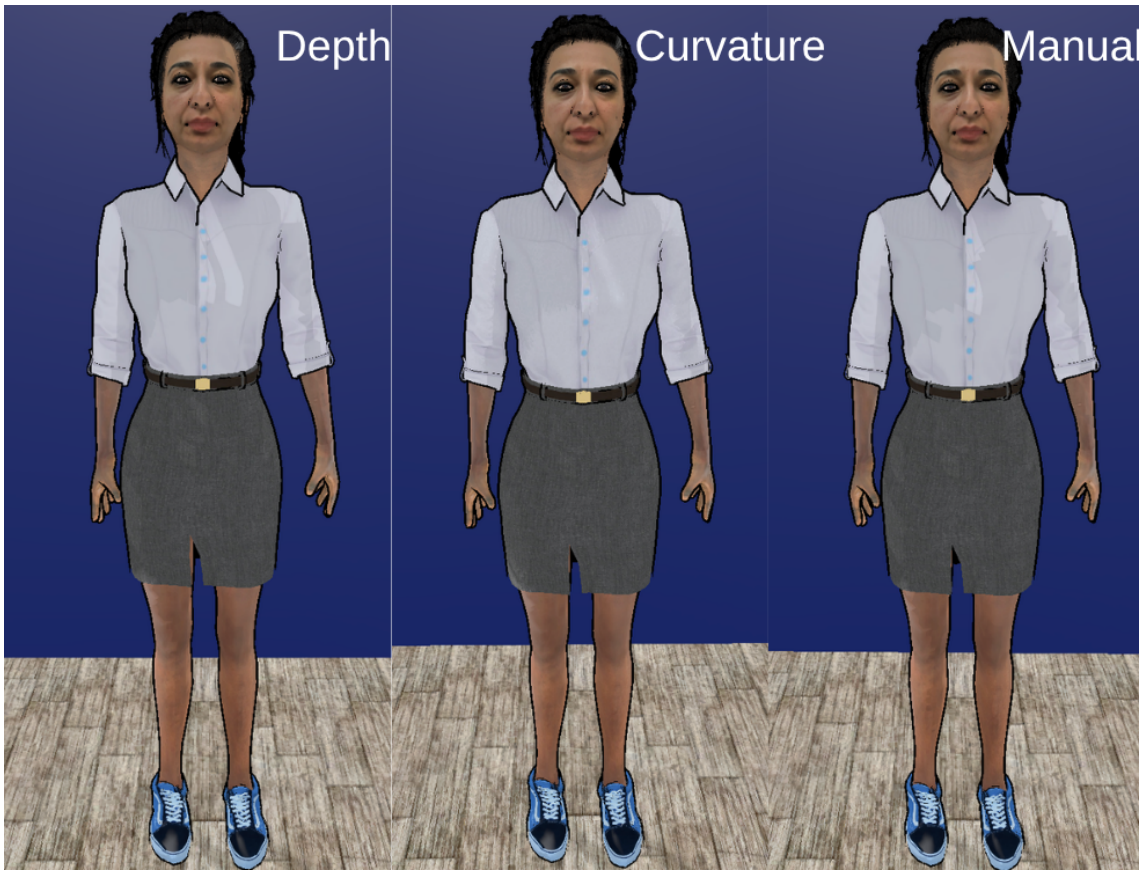
a region toward a common direction and produces coherent flat banding that reinforces the depth-driven abstraction.

The U axis encodes shadow-attenuated cosine irradiance against the possibly abstracted surface normal, remapped to $[0, 1]$ to align with standard ramp-texture conventions. All three platform implementations obtain the same shadow-attenuated directional light signal; the platform-specific derivation is described in Section 5.4.2. A specular highlight is added over the ramp result using the Blinn-Phong half-vector model [21], which evaluates surface shininess through the dot product of the surface normal and the halfway direction between the light and viewer; the size and softness of the resulting bright dot are independently tunable.

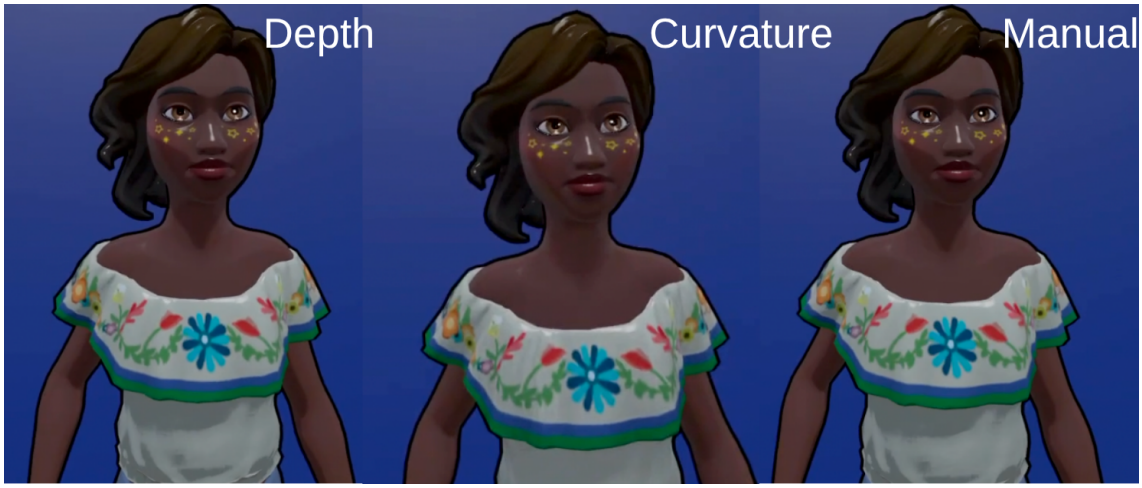
Across all three avatar platforms, the three V-axis modes — depth, curvature, and manual — did not produce strongly distinct visual results; the depth-based and manually configured variants yielded comparable shading appearances, and curvature mode showed similarly limited differentiation. Compared to V1, X-Toon produced a softer, more abstracted tonal read overall, with lighting bands that dissolved toward a unified mid-tone rather than snapping between discrete steps; X-Toon was therefore selected as the default toon shading variant. Like V1, neither technique produces any edge signal: surface boundaries, clothing transitions, and texture-defined detail remain unmarked. This shared limitation motivates the introduction of edge detection from V2 onward.



(a) Jody (Mixamo): depth mode (left), curvature mode (centre), manual mode (right). In depth mode the U coordinate is compressed toward the ramp midpoint as camera distance increases; in curvature mode abstraction rises on flat surface regions; in manual mode the abstraction level is held constant. The abstract normal map is active in all three variants.



(b) Avaturn: the same three modes applied to the photograph-reconstructed avatar. The denser PBR normal map produces stronger band differentiation than Jody across all three variants.



(c) Meta SDK: depth, curvature, and manual modes computed within the post-lighting hook on the composited PBR output. Geometric normal smoothing replaces the abstract normal map; shadow and ambient contributions are pre-composited by the SDK renderer.

Figure 4.7: V1.2 (X-Toon Extended Toon Shading) across all three avatar platforms, comparing depth-driven (left), curvature-driven (centre), and manually set (right) abstraction modes. Each variant controls the V axis of the 2D ramp, determining how tonal bands flatten and band-edge transitions soften as the abstraction level rises (Equations 4.3–4.4).

4.3.4 V2 — Screen-Space Normal Edge Detection

V2 introduces the first interior edge signal by detecting discontinuities in the interpolated surface normal across adjacent screen pixels. Because the detection operates in screen space, the GPU hardware intrinsics `ddx` and `ddy` expose the finite difference of any fragment-stage value across a 2×2 pixel quad, requiring no additional texture samples. This zero-sample-cost property is a deliberate design constraint for Quest VR, where GPU fragment-shader budget is limited and any extra texture fetch in a per-avatar pass directly reduces the sustainable avatar count. Applying these derivatives to the world-space surface normal yields the gradient magnitude defined in Section 3.2.2, which responds to creases, mesh boundaries, and regions of high curvature — boundaries that are invisible to the toon shading of V1 and V1.2. The toon quantisation from V1 is retained as the surface shading base, with the detected edge mask $E \in [0, 1]$ composited over it:

$$C_{\text{final}} = (1 - E)\, C_{\text{shaded}} + E\, C_{\text{edge}} \tag{4.7}$$

where C_{edge} is a configurable line colour. A continuous smoothstep function converts the gradient magnitude into E rather than a binary step, because a hard threshold causes a whole surface patch to snap between zero and full edge weight in a single frame when the surface normal rotates past the detection boundary during animation. The smoothstep blends this transition over a configurable half-width σ, suppressing the temporal pop.

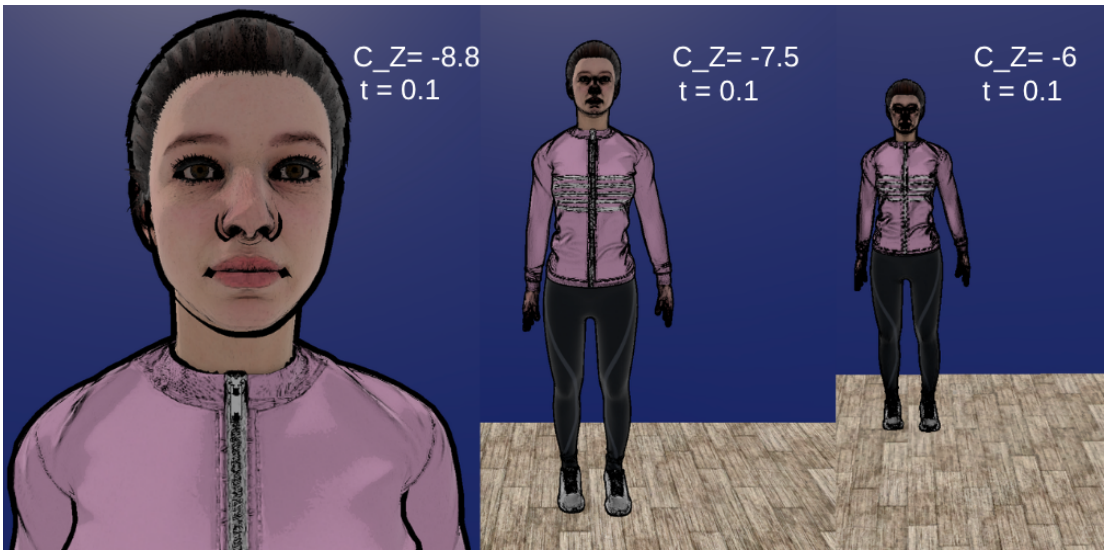
V2 detects rapid changes in surface orientation: wherever the normal gradient magnitude exceeds the threshold, an interior crease line is drawn, marking features such as the brow ridge, jaw contour, and clothing fold boundaries.

Visual behaviour varies across platforms because the richness of the normal data available differs. On Jody (Mixamo), the normal field encodes both large-scale mesh curvature and the detail baked into the PBR normal map, producing a moderately dense interior edge pattern. On Avaturn, the photogrammetry-derived normal map carries finer microdetail, yielding a denser pattern on the face. On the Meta SDK platform, the SDK's composited fragment normal already incorporates the SDK's own normal map through its internal PBR pipeline, without requiring any additional decode step. These differences are purely a function of the quality of normal-map data on each platform, not of the algorithm itself.

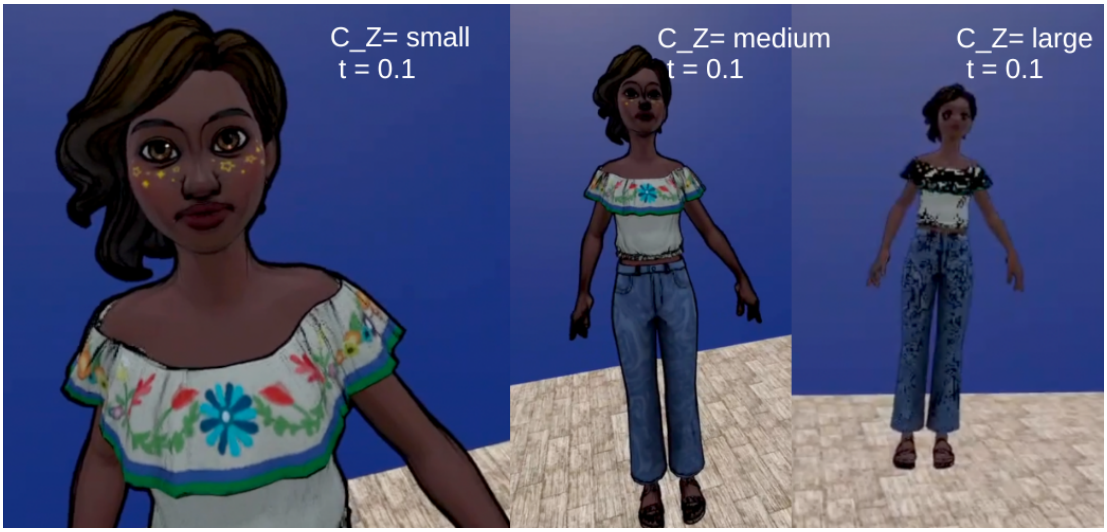
The technique has two limitations. First, edge thickness is not independent of viewing distance: as the avatar recedes, each screen-space quad subtends a larger solid angle of the surface, causing the apparent gradient magnitude to increase and edges to spread rather than thin. This artefact was directly observed during development. Second, the technique is blind to colour: a clothing pattern boundary or a painted texture transition on a smoothly curved surface produces no normal discontinuity and therefore no edge line. Both limitations motivate the shift to UV-space gradient computation in V3.

Figure 4.8 documents the distance artefact across all three platforms. Each row shows three columns at progressively greater viewing distances. C_Z is the world-space Z coordinate of the Unity Editor camera at capture time; for Jody and Avaturn the camera transform was positioned programmatically, so exact C_Z values were recorded. The Meta SDK avatar is

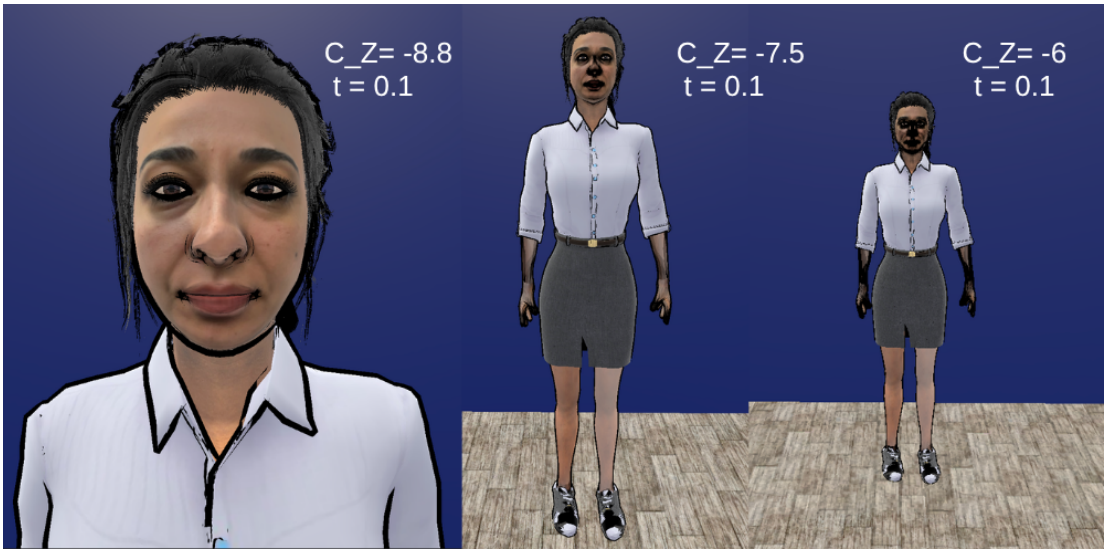
rendered inside the Quest VR headset, where the viewing position is the physical headset pose tracked in room space; this coordinate does not share the same world-space axis as the Unity Editor camera, so screenshots were taken at approximate distances and labelled relative to one another (small, medium, large) rather than with a recorded C_Z value.



(a) Jody (Mixamo): C_Z is the Unity Editor camera world-space Z coordinate; $t = 0.1$ (edge threshold).



(b) Meta SDK: camera distance is the physical headset pose and cannot be expressed as a comparable C_Z value; distances are labelled small, medium, and large. $t = 0.1$.



(c) Avaturn: same C_Z positions as Jody; denser edge pattern due to the photogrammetry-derived normal map. $t = 0.1$.

Figure 4.8: V2 (Screen-Space Normal Edge Detection) across all three platforms at three viewing distances. Edge thickness increases with distance in all rows, demonstrating the characteristic distance artefact of screen-space derivative computation.

4.3.5 V3 — Sobel Edge Detection

V3 shifts the gradient computation from screen space to UV/texture space, addressing both the distance-dependence and the selectivity limitation of V2. The Sobel operator is applied to the avatar's diffuse albedo texture to extract colour-based interior edges that have no counterpart in the surface normal field, capturing clothing pattern boundaries, facial texture transitions, and painted surface detail invisible to V1 through V2.

The operator samples the albedo at nine positions arranged in a 3×3 grid centred on the current fragment's UV coordinates, converting the RGB samples to scalar luminance values before convolution. The horizontal and vertical gradient components G_x and G_y are estimated using the Sobel kernels defined in Equation 3.2 [7]; the gradient magnitude $M_S = \sqrt{G_x^2 + G_y^2}$ provides a scalar edge-strength estimate at each fragment. Fragments exceeding a configurable threshold are classified as edge pixels; the resulting mask is composited over the toon-shaded surface colour following Equation 4.7.

UV seam boundaries require a targeted countermeasure: texels straddling a seam carry UV coordinates that are distant in texture space but adjacent on screen, so the colour difference registers as a strong gradient with no visual counterpart. This is suppressed by a luminance coherence gate that measures the spread of luminance values across the nine sampled positions; detections where the spread exceeds a rejection limit are discarded, since genuine structural edges produce moderate tonal contrast while seam spikes span nearly the full luminance range in a single step. A second upper bound clips the overall edge magnitude, rejecting any detection whose strength exceeds the plausible range of a surface-material boundary.

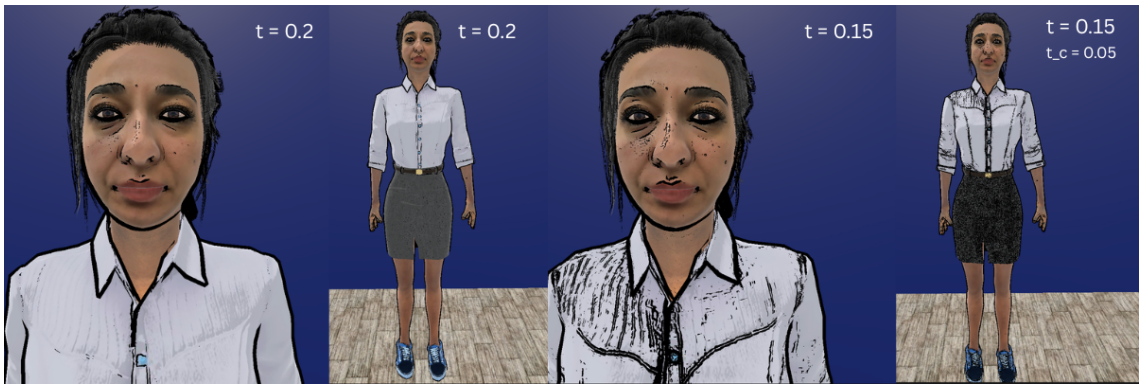
The three platforms differ in what the operator samples and how the countermeasures apply. On Mixamo (Jody), the flat painted albedo has well-separated colour regions, so the operator detects garment boundaries and pattern transitions cleanly. Avaturn's texture is derived from user-uploaded selfie photographs, which contain baked-in illumination from the capture environment. This baked lighting creates luminance gradients within regions that should appear uniform, causing the Sobel operator to respond to photographic skin shading rather than structural material boundaries. The nose bridge is particularly affected: the highlight and shadow variation across the ridge registers as a strong gradient even at moderate threshold values, producing a visible artefact that does not appear on Jody or the Meta SDK avatar. On the Meta SDK, only the composited PBR colour is accessible rather than the raw albedo; the operator responds to shadow boundaries and specular transitions in addition to material boundaries, which reduces selectivity but avoids the baked-illumination sensitivity of the photographic source.

The detection threshold t required to isolate structural edges varies substantially across platforms. On Jody, $t = 0.1$–0.2 balances edge coverage against noise; lowering t below 0.1 reveals additional detail such as eyebrow edges and clothing panel seams, but amplifies low-contrast skin gradients. On Avaturn, the same threshold range produces stronger artefacts due to the baked illumination in the photographic albedo: as shown in Figure 4.9, even at $t = 0.2$ the nose bridge registers a visible Sobel response. Clothing regions, however, tolerate a lower threshold ($t = 0.15$) without comparable artefacts, since edge

responses on fabric read as intentional texture detail rather than skin noise. Applying a material-specific threshold — higher on the face to suppress the illumination artefact, lower on clothing to reveal fabric detail — therefore produces a more plausible abstracted appearance than any uniform global value. The Meta SDK avatar requires a substantially lower threshold ($t = 0.02$–0.05) because the PBR-composited signal has lower per-pixel gradient magnitude at material boundaries; a single threshold applies uniformly across all submeshes since per-region control is not available through the hook.

A further limitation shared by all three platforms is sensitivity to texture noise: any colour gradient, whether from structural features or fine-scale variation and compression artefacts, contributes to the edge signal. V3.2 addresses this by pre-smoothing the texture input before differentiation, following Canny's established principle [8] that gradient-based detectors require prior smoothing to suppress noise sensitivity.

(a) Jody (Mixamo): $t = 0.2$ (left pair) and $t = 0.1$ (right pair). Lower threshold reveals eyebrow and seam detail but amplifies skin gradients.



(b) Avaturn: $t = 0.2$ (left pair); per-region $t = 0.2$ face, $t_c = 0.15$ clothing (right pair). The nose bridge artefact (visible at both thresholds) results from baked illumination in the photographic albedo. Lower clothing threshold reveals fabric detail.

(c) Meta SDK: $t = 0.02$ (left pair) and $t = 0.05$ (right pair). Lower thresholds are required because Sobel runs on the PBR-composited output rather than the raw albedo; per-region threshold control is not available.

Figure 4.9: V3 (Sobel Edge Detection) across the three avatar platforms. The threshold required to isolate structural edges differs substantially across platforms due to differences in texture source and gradient magnitude.

4.3.6 V3.2 — Gaussian Prefiltered Sobel

V3.2 extends V3 by placing a Gaussian pre-filter before the Sobel gradient computation and replacing the explicit seam-rejection gate with a configurable threshold band and a multi-pass edge-sharpening pipeline. The smoothing motivation is grounded in the analysis of Section 3.2.3: both Canny [8] and Marr and Hildreth [9] establish that gradient-based

edge detection requires prior low-pass smoothing, because differentiation is inherently a high-pass operation that amplifies noise and fine texture variation alongside genuine intensity boundaries. Without a smoothing step, the nine-sample Sobel neighbourhood of V3 responds to compression artefacts and skin microdetail in the same way it responds to clothing seams or facial contour transitions. The Gaussian filter is the preferred smoothing choice because it is isotropic: it attenuates noise equally in all spatial directions without introducing an orientation bias that would systematically skew the distribution of detected edge directions [9].

The pre-filter is a 9-tap weighted sum parameterised by three explicit coefficients: a centre weight, a cardinal weight shared by the four axis-aligned neighbours, and a diagonal weight shared by the four corner neighbours. The defaults follow the classical discrete 3×3 Gaussian approximation [7]: centre $= 0.25$, cardinal $= 0.125$, diagonal $= 0.0625$, corresponding to kernel entries $4/16$, $2/16$, and $1/16$ respectively. All three coefficients are normalised to unity before use, so the filter remains a valid weighted average regardless of individual parameter adjustments. Setting the cardinal and diagonal weights to zero collapses the filter to a point sample, reproducing plain V3 behaviour; increasing them widens the smoothing footprint without requiring formula reparametrisation. This explicit parameterisation was chosen to give direct authoring control during cross-avatar tuning, where different texture sources benefit from different amounts of pre-filtering.

Because the pipeline executes within a single rendering pass, the Gaussian-smoothed texture cannot be stored in an intermediate render target for the Sobel stage to read from. The 9-tap smoothing must therefore be recomputed independently at each of the nine Sobel neighbourhood positions, yielding up to $8 \times 9 = 72$ texture samples per fragment when the pre-filter is fully enabled, compared to eight unsmoothed samples in V3.

The thresholded gradient magnitude passes through four successive smoothstep operations before the edge mask is finalised. Each pass applies a smoothstep centred at 0.5 with a half-width that shrinks as a single tightness parameter increases: at zero tightness each half-width equals 0.5, making every pass a linear pass-through; at full tightness the four widths narrow to 0.03, 0.15, 0.25, and 0.35 respectively, composing into a steep sigmoid that approaches a binary threshold without the temporal aliasing of a `step` call. A final power curve remaps the stacked output: exponents greater than one suppress the soft halo around strong edges; exponents less than one widen thin lines.

The principal limitation of this approach is that Gaussian smoothing is indiscriminate: it attenuates structural edges and artefact gradients in equal measure, because it operates on luminance values without semantic knowledge of which gradients are visually meaningful. Edge coverage and noise suppression cannot therefore be independently optimised; attenuating one necessarily attenuates the other. Testing confirmed this: the pre-filter reduced spurious detections at lower thresholds, but simultaneously softened genuine structural lines, producing cleaner edges in aggregate rather than more selective ones.

Applying an aggressive Gaussian pre-filter while keeping the same threshold values as V3 produced a mixed outcome. Some false detections around compressed skin gradients were attenuated, but the pre-filter also softened genuine structural boundaries, causing

fine edge lines visible in V3 to disappear. The noise reduction did not resolve threshold sensitivity; it shifted the trade-off from false positive edges to false negatives along low-contrast boundaries. Among the three platforms, Avaturn produced the weakest results under both V3 and V3.2. Its selfie-derived texture bakes camera-captured illumination directly into the albedo, generating large gradient magnitudes across the skin surface that are unrelated to geometric structure. Gaussian smoothing reduced these spurious gradients only partially, and the remaining signal still dominated the Sobel response at the threshold values used, making meaningful structural edge detection unreliable on this platform.

(a) Jody (Mixamo): aggressive Gaussian pre-filter with $t = 0.2$ (left pair) and $t = 0.1$ (right pair). Skin compression artefacts are partially reduced relative to V3, but some structural edges are attenuated by the smoothing.

(b) Meta SDK: $t = 0.02$ (left pair) and $t = 0.05$ (right pair). Pre-filtering reduces high-frequency false detections at the cost of fine edge loss; the overall edge pattern is sparser than V3 at matching thresholds.

Figure 4.10: V3.2 (Gaussian Prefiltered Sobel) on Jody and the Meta avatar. Avaturn is omitted; under both V3 and V3.2 its selfie-derived albedo generates pervasive skin gradients that overwhelm the structural edge signal, producing the least meaningful results across the three platforms.

4.3.7 V4 — Halftone Screening and Cross-Hatching

V4 introduces two tone-responsive mark-making techniques, halftone dot screening and cross-hatching, which represent a conceptual departure from the edge-line vocabulary of V1 through V3.2. Rather than detecting boundaries and compositing contour lines over the surface, both techniques encode surface tone as the spatial density of ink marks: darker regions receive denser marks, lighter regions sparser ones. No explicit edge signal is produced; the impression of form emerges from the tonal organisation of the mark field.

The hatching component is motivated by the tonal art map (TAM) framework of Praun et al. [12], which proposes a systematic method for rendering hatching strokes in real time by organising them into a 2D grid of pre-rendered images indexed by level of detail and tone. The defining structural property of a TAM is stroke nesting: every stroke visible at a lighter tone also appears in all darker tone images, so a transition toward deeper shadow adds new stroke layers rather than substituting one pattern for another [12]. This incremental accumulation produces the visual coherence of hand-drawn cross-hatching across a range of tones.

The implementation in this thesis approximates the TAM structure procedurally, generating four analytically computed line layers directly in the fragment shader rather than sampling a pre-rendered texture stack. A primary-direction layer activates at moderate shadow values; a cross-direction layer adds perpendicular strokes at deeper shadow; a denser diagonal layer fills progressively darker regions; and a solid-fill layer covers near-black areas. These four layers correspond conceptually to the tone columns of a TAM, but require no offline construction phase and no texture memory, since each layer is computed analytically from the fragment's UV coordinates and tone value.

The halftone component generates a regular dot grid in UV space, rotated by a configurable angle. The dot radius at each grid cell scales with the square root of the local darkness value. This square-root relationship follows the standard photomechanical halftone model, where dot area is proportional to ink coverage: equal-area dots at full tone shrink to zero at full light, keeping perceived density proportional to darkness throughout the tonal range.

The tone value driving both components is derived differently across platforms. On Mixamo and Avaturn, where the full URP forward pass is available, tone is computed from the Lambertian dot product attenuated by the scene shadow map, consistent with the lighting computation in V1 through V3.2. On the Meta SDK platform, the hook executes after all PBR lighting has been composited; tone is derived from the ITU-R BT.601 luminance of that composited value, which already incorporates shadow, ambient, and subsurface contributions, so mark density responds correctly to the full material response.

Both components share an ink and paper colour model consistent with the blending approach used across the technique series. A configurable paper colour provides the unlit base, an ink colour provides the mark, and an optional albedo tint blends each toward the surface texture, allowing marks to reference the underlying material. A global strength parameter governs how strongly the mark pattern is composited over the original shaded surface.

The defining limitation of V4 relative to V2 and V3 is the complete absence of geometric contour information: mark density responds to tone, not to surface boundaries. Facial contour lines, clothing seams, and texture transitions produce no direct mark response. V4 is most legible on avatars with strong tonal variation from directional lighting; on uniformly lit surfaces the mark field carries little structural reading of form.

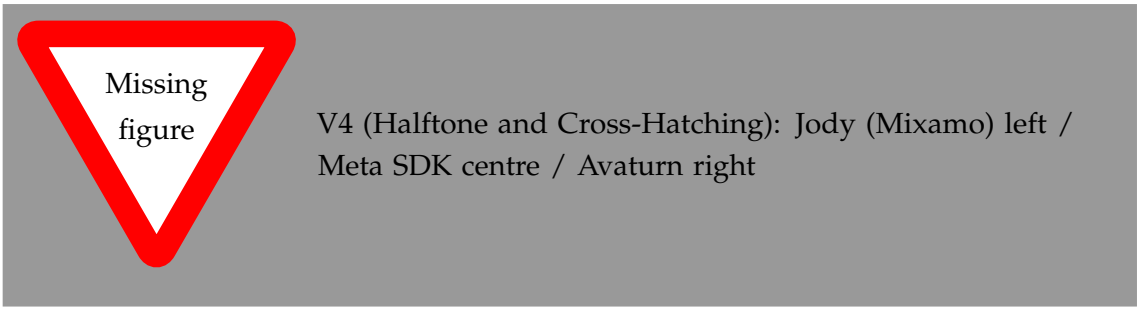


Figure 4.11: V4 (Halftone Screening and Cross-Hatching) across the three avatar platforms. Jody and Avaturn compute tone from the shadow-attenuated Lambertian dot product; the Meta SDK derives tone from the ITU-R BT.601 luminance of the composited PBR output.

4.3.8 V5 — Hierarchical Edge Detection with Multiple Cues

The sixth technique fuses three independent edge cues through weighted combination, addressing the limitation that no single cue reliably captures all visually meaningful boundaries. A depth proximity layer uses derivatives in screen space of the camera-to-fragment distance to detect silhouette-level boundaries. A normal layer detects surface crease edges from normal discontinuities. A colour layer applies the Roberts Cross operator [7] to the diffuse texture to capture painted detail boundaries. The three layers are blended using weights per layer that can be adjusted at runtime, allowing the visual balance between silhouette, crease, and texture edges to be controlled interactively. This technique detects the broadest range of edge types of any approach developed to this point.

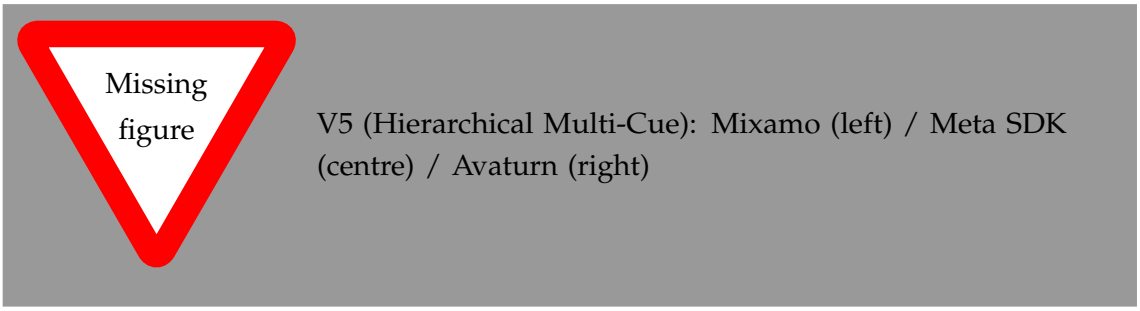


Figure 4.12: V5 (Hierarchical Edge Detection with Multiple Cues) across the three avatar platforms.

4.3.9 V6 — Kuwahara Painterly Filter

Rather than drawing explicit edge lines, the seventh technique applies the isotropic Kuwahara filter to the rendered surface colour to produce a painterly abstraction. For each fragment, four overlapping quadrant regions are sampled; the output is set to the mean colour of whichever quadrant has the lowest local variance. Uniform surface regions are smoothed toward a flat painterly fill, while region boundaries are sharpened implicitly because the selected quadrant is always the one that falls cleanly on one side of the boundary. This is the only technique in this thesis that does not produce explicit ink lines; the abstraction operates entirely through regional colour smoothing.

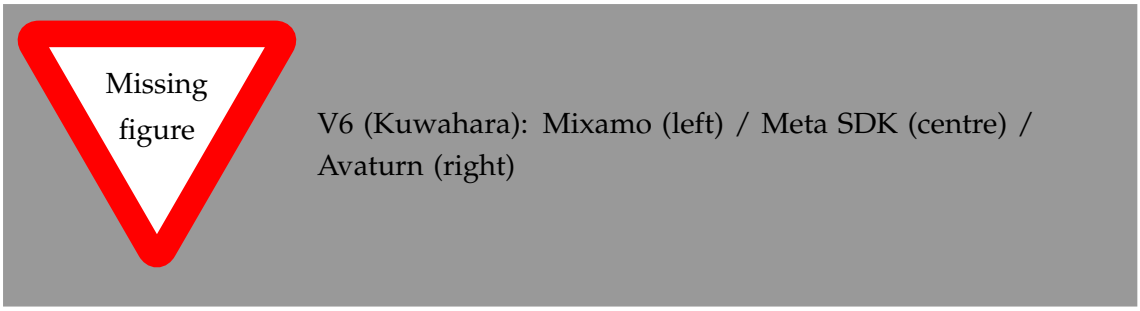


Figure 4.13: V6 (Kuwahara Painterly Filter) across the three avatar platforms.

4.3.10 V7 — Kuwahara with Sobel Edges

The eighth technique combines the painterly region abstraction of V6 with the ink-line edges of V3. The Kuwahara pass is applied first to produce smooth filled regions, and Sobel edges are composited on top to add explicit contour lines. The result is a comic-book aesthetic: flat coloured regions separated by drawn boundaries. Because the edge detection runs on the abstracted Kuwahara output rather than the original diffuse texture, the lines reflect the abstracted region structure rather than every fine texture variation.

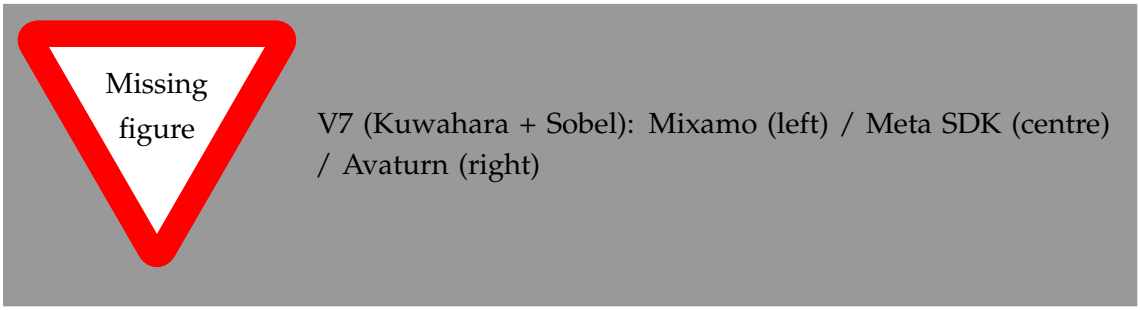


Figure 4.14: V7 (Kuwahara with Sobel Edges) across the three avatar platforms.

4.3.11 V8 — Composite: Kuwahara, Gaussian Sobel, and Hierarchical

The ninth technique chains all three main abstraction strategies into a single pipeline. The Kuwahara filter is applied first to produce the painterly base. Gaussian prefiltered Sobel edges are then composited to add smoothed colour boundary lines. Finally, the hierarchical multi-cue layer contributes depth, normal, and colour edges with independent weighting. This produces the richest visual output of any technique in the set, combining surface abstraction with two complementary edge detection strategies at the cost of the highest sample count per fragment of all techniques developed so far.

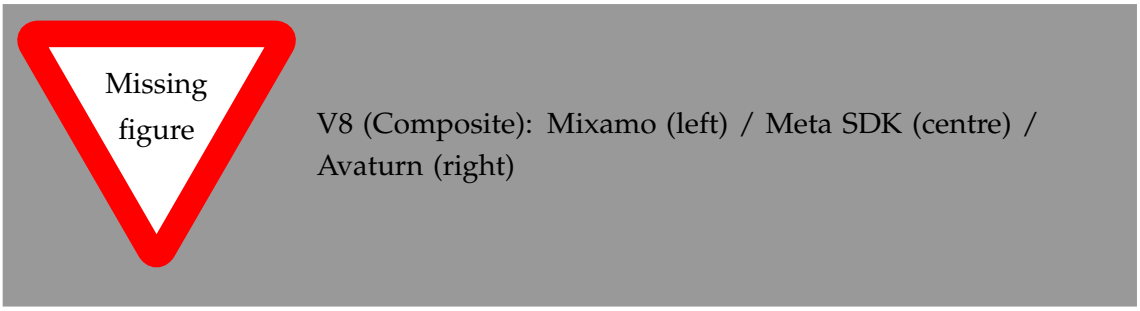


Figure 4.15: V8 (Composite: Kuwahara, Gaussian Sobel, Hierarchical) across the three avatar platforms.

4.4 Technique Selection for the User Study

Following the iterative development phase, a single technique is selected for the NPR condition in the user study based on visual assessment across all three avatar platforms. Three criteria guide the selection. First, facial expression detail must remain readable under the stylisation, since the study measures trust responses to a speaking avatar. Second, the effect must be visually coherent across viewing distances, since the video scenarios are filmed at a fixed conversational distance. Third, the abstraction must be clearly distinct from the photorealistic baseline to ensure that any difference in trust ratings reflects a genuine perceptual response to the stylisation. The outcome of this assessment is documented in Chapter 6.

4.5 User Study Design

4.5.1 Objective and Hypotheses

The user study addresses RQ3 by measuring how the visual representation of a speaker affects the perceived trustworthiness of their advice in a concrete decision situation. Three presentation conditions are compared: a real human speaker, a photorealistic Avaturn avatar, and an Avaturn avatar rendered with NPR stylisation. Two hypotheses are tested.

H1: Participants will rate the real-person condition as most trustworthy, establishing a human baseline against which avatar conditions can be compared.

H2: The NPR-stylised avatar will not be rated significantly less trustworthy than the photorealistic avatar, and may be rated more trustworthy if the photorealistic condition triggers uncanny valley discomfort.

4.5.2 Conditions

The study compares three presentation conditions:

C0 — Real Person: A video recording of a real human speaker delivering the recommendation. This serves as the trust ceiling baseline.

C1 — Photorealistic Avatar: The same speaker scenario rendered using a photorealistic Avaturn avatar with standard PBR shading. Facial expressions are driven by blendshape animation to match the delivery timing.

C2 — NPR Avatar: The Avaturn avatar rendered with the NPR technique selected from the visual assessment as providing the best balance of abstraction and expressiveness.

4.5.3 Scenarios

Three video scenarios are designed to present naturalistic decisions in which the speaker's trustworthiness is relevant. Each scenario introduces a situation with two plausible options and shows the speaker recommending one. The scenarios are kept domain-neutral to avoid prior knowledge effects. After watching each video, participants rate how trustworthy they found the speaker's recommendation.

Scenario content, option framing, and speaker delivery are held constant across conditions; only the visual presentation of the speaker changes. The three scenarios are counterbalanced across conditions using a Latin square to control for scenario-condition order effects.

4.5.4 Procedure and Measures

The study uses a within-subjects design. Each participant watches all three conditions across the three scenarios, with condition-scenario assignment randomised per participant. After each video, participants complete a short questionnaire including a trust rating scale and items from the Godspeed questionnaire [14] assessing human-likeness and eeriness.

Within-subjects differences across the three conditions are analysed using a repeated-measures design. Pairwise comparisons between each avatar condition and the real-person baseline, and between the two avatar conditions, are conducted with Bonferroni correction. Effect sizes are reported as partial η^2.

4.5.5 Ethical Considerations

The study was reviewed and approved by the University of Konstanz ethics committee. Participants provided written informed consent before participation. Data is anonymised at collection; no personally identifiable information is stored alongside questionnaire responses. Participation was voluntary with the right to withdraw at any time.

5 Implementation

This chapter documents the technical realisation of the NPR avatar system. It covers the development environment, the integration path for each avatar platform, the implementation of the shared rendering structure, and the HLSL-level details of each stylisation technique developed to date. Runtime tooling developed to support iterative development is described at the end of the chapter.

5.1 Development Environment

The system was developed in Unity 6 (version 6000.0.60f1) using the Universal Render Pipeline (URP). The target hardware is the Meta Quest family of standalone VR headsets. The Meta Avatars SDK version 40.0.1 was used for the SDK-managed avatar platform. All shader code is written in HLSL and targets the URP forward rendering path.

5.2 Avatar Platform Integration

5.2.1 Mixamo

Mixamo characters are exported as FBX assets and imported into Unity as standard skinned mesh renderers. No proprietary shader system is involved, so all URP materials can be replaced directly with custom NPR shaders without restriction. The Mixamo character Jody served as the primary prototyping target during technique development, allowing the NPR shaders to be iterated freely before the more constrained platform integrations were addressed.

5.2.2 Meta Avatars SDK

The Meta Avatars SDK manages avatar materials through its Extensible Shader Framework (ESF), which generates platform-specific HLSL from parameterised templates. The generated source is not intended to be edited directly. Custom logic is instead injected through named hook functions that the ESF exposes at defined points in the shader pipeline.

The hook used in this thesis is `AppSpecificPostManipulation`, which executes per fragment after all SDK PBR lighting — subsurface scattering, eye glints, hair shading — has been computed. It receives the accumulated lit colour and returns a modified colour, giving full control over the final fragment output without disrupting any SDK rendering logic. The hook is defined across three files that the ESF includes automatically during shader compilation:

- `app_functions.hlsl` — the body of `AppSpecificPostManipulation`; dispatches to the active NPR technique based on the `_NPREffect` keyword

- `app_declarations.hlsl` — uniform declarations for all NPR parameters, shared across techniques
- `app_variants.hlsl` — shader feature keyword declarations that activate each technique branch

A further complication is the SDK's LOD management system, which replaces avatar materials automatically as camera distance changes. This reverts any custom shader assignment made at load time. The `AvatarShaderSwapper.cs` script addresses this by running a periodic coroutine that re-applies the custom NPR materials after each LOD transition is detected. The script also handles the initial material replacement, waiting for the asynchronous avatar load to complete before copying base colour and normal textures from the SDK materials into the custom shader properties.

5.2.3 Avaturn

Avaturn exports avatars in the `.glb` format, the binary container of the glTF 2.0 standard. Unity does not natively support runtime glTF loading, so the GLTFast package is used to load the asset at runtime. On load completion, GLTFast produces a standard Unity GameObject hierarchy with skinned mesh renderers and PBR materials assigned from the glTF material definitions, including albedo, normal, and metallic-roughness textures. Because Avaturn avatars carry no proprietary shader constraints once imported, the custom NPR materials are assigned to the resulting skinned meshes using the same approach as Mixamo: direct material replacement after load. Blendshape animation targets exported by Avaturn are preserved through the GLTFast import and driven at runtime via `SkinnedMeshRenderer.SetBlendShapeWeight` for the video recording scenarios used in the user study.

5.2.4 Common Scene Environment for Cross-Avatar Comparison

A key requirement for meaningful cross-avatar comparison is that any visible differences between the Jody (Mixamo) and Avaturn avatars arise from the NPR technique itself, not from incidental differences in the rendering environment. To satisfy this, both avatars are placed in the same Unity scene and rendered under identical environmental conditions.

A single directional light is used as the primary illumination source for both avatars. Its transform — rotation, intensity, and colour temperature — is shared: the light GameObject exists once in the scene hierarchy and both avatar GameObjects fall within its influence. The light direction is set to a front-upper-left angle (-30° elevation, -30° azimuth from the avatar forward vector), which is a standard portrait lighting configuration that reveals facial structure and clothing wrinkles without casting extreme shadows. Intensity is fixed at 1.0 lux with a neutral white colour so that no colour bias is introduced by the light source.

Both avatars stand in the same physical room geometry. The environment is a studio-style enclosure with a neutral grey floor and walls, a controlled skybox, and no emissive surfaces, ensuring that indirect lighting contributions — sampled from spherical harmonics

in the URP forward path — are identical for both avatars. Ambient intensity and the GI contribution are baked from the shared skybox, so the ambient component of the lighting equation is the same regardless of which avatar is being evaluated.

This configuration means that when a researcher or participant views the Jody and Avaturn avatars side by side or switches between them, all differences in shading, edge quality, and tonal response are attributable to the technique and its interaction with each avatar's specific mesh and texture assets, rather than to environmental factors. It also ensures that the scalar parameter values tuned for one avatar — outline width, toon band count, shadow strength — have the same perceptual effect on the other.

5.3 Shared Rendering Structure

The NPR shader used across Mixamo and Avaturn is `Avatar-Meta-UGB.shader`, a multi-pass URP shader containing two passes: `NPROutline` and `ForwardLit`. For the Meta SDK platform, the forward-lit NPR logic runs inside `AppSpecificPostManipulation`; the outline pass is handled by a separate material applied to a duplicate mesh layer.

The core forward-lit dispatch lives in `Style2MetaAvatarCore.hlsl`. It reads the active `_NPREffect` shader keyword and branches to the corresponding technique include file. This structure means all techniques share the same entry point and parameter infrastructure; switching techniques at runtime requires only toggling the active keyword via `Material.EnableKeyword` and `Material.DisableKeyword`, which the runtime UI performs without recompilation.

5.3.1 Inverted Hull Outline Pass

The outline pass implements the inverted hull silhouette. In the Mixamo and Avaturn shaders the pass sets `Cull Front`, `ZWrite On`, `ZTest Less`; in the Meta SDK shader it sets `Cull Front`, `ZWrite Off`, `ZTest LEqual` to avoid conflicting with the SDK's depth compositing. The vertex shader displaces each vertex outward along its world-space normal by `_OuterOutlineWidth` (Mixamo/Avaturn, world-space metres) or `_OutlineWidth * 0.001` (Meta, converted from UI units). The fragment outputs a flat `_OuterOutlineColor` or `_OutlineColor`. An `_OutlineDepthBias` property shifts the clip-space Z coordinate toward the camera to prevent z-fighting where the expanded shell and the original surface meet at shallow angles.

5.3.2 Alpha Cutout Handling

Alpha cutout is controlled by the `_EnableAlphaTest` flag rather than being always-on. When enabled, both the outline pass and the forward-lit pass sample the base colour texture alpha channel and call `clip(alpha - threshold)` before any displacement or shading is applied. In the outline pass this ensures the expanded hull is discarded wherever the hair card texture is transparent, preventing a rectangular border from appearing around each card. The cutout threshold is exposed in the runtime UI and

adjusted per avatar submesh — a lower threshold is typically used for eyelashes (dense geometry) than for hair cards (sparse, large transparent regions).

This explicit alpha test is necessary on the Mixamo and Avaturn platforms because their outline pass is a minimal, self-contained `Cull Front` pass with no knowledge of the surface material's transparency. Without it, the inverted hull is extruded over the full rectangular hair card mesh, including its fully transparent regions, producing a visible rectangular border around each card.

The Meta Avatar SDK outline pass does not require this mechanism. The outline is implemented as a named pass inside the same multi-pass shader that handles the surface, using the same `Style2MetaAvatarCore.hlsl` include as the lit pass. The SDK's fragment pipeline — including its own internal alpha handling for hair and eyelash geometry — runs to completion before the `AppSpecificPostManipulation` hook is invoked. Any fragment that the SDK would discard as transparent is already clipped before the hook's `o.color = _OutlineColor` line is reached. Alpha cutout is therefore subsumed by the SDK and requires no additional shader property or UI control on the Meta platform.

(a) Jody — alpha test off **(b)** Jody — alpha test on **(c)** Avaturn — alpha test off **(d)** Avaturn — alpha test on

Figure 5.1: Effect of alpha cutout on the inverted hull outline for Jody (Mixamo) and Avaturn avatars. Without alpha test (left of each pair) the outline is extruded over the full rectangular hair card mesh, producing a visible border around transparent regions. Enabling `_EnableAlphaTest` discards outline fragments below the alpha threshold, confining the hull to the opaque silhouette.

5.4 NPR Technique Implementation

All shader parameters described in the following subsections are accessible through two routes. The primary route is the Unity Inspector: each parameter appears as a labelled property field on the material asset and can be adjusted directly in the Editor during offline development. The secondary route is the in-VR WorldSpace UI panel described in Section 5.5.1, which exposes the same properties as interactive sliders and toggles inside the running application on the headset. Both routes write to the same underlying `MaterialPropertyBlock` values; changes made through either interface take effect immediately without shader recompilation. Technique switching — enabling or disabling an NPR variant — is performed through a dropdown row in the in-VR panel, which calls `Material.EnableKeyword` and `Material.DisableKeyword` on the active material set.

5.4.1 V1 — Toon Shading

Toon shading — also known as cel shading — replaces the continuous Lambertian diffuse term with a discrete, stepped function. In conventional diffuse shading, surface brightness varies smoothly according to $\mathbf{N} \cdot \mathbf{L}$, the dot product of the surface normal and light direction, producing smooth gradients that convey three-dimensional form. Toon shading instead partitions this continuous lighting value into a fixed number of discrete bands, producing the characteristic flat visual style where shadows and highlights appear as distinct regions of uniform colour rather than blended gradients [5]. A smoothstep transition is applied at each band boundary to soften the step and suppress temporal aliasing during animation. A shadow strength parameter additionally controls the contrast between illuminated and shadowed regions by interpolating between the fully quantised value and constant full brightness, allowing the depth of the banded effect to be tuned independently of the band count.

Three separate shader implementations exist for V1, one per avatar platform. The outline pass is structurally identical for Mixamo and Avaturn (inverted hull, normal-offset displacement, explicit alpha cutout); the Meta variant shares the same inverted hull geometry but omits the explicit alpha test because the SDK's fragment pipeline handles hair and eyelash transparency internally before the outline hook fires (see Section 5.3.2). The toon shading pass differs because the Mixamo and Avaturn variants operate on raw $\mathbf{N} \cdot \mathbf{L}$, while the Meta variant operates on the composited PBR output.

Mixamo

Shader: `Assets/Shaders/JodyNPRShaders/V1_ToonShading_GeometryOutline.shader`
Pipeline target: URP, `#pragma target 3.0`

The outline pass sets `Cull Front`, `ZWrite On`, `ZTest Less` and displaces each vertex along its world-space normal by `_OuterOutlineWidth` (world units). The fragment returns a flat `_OuterOutlineColor`. When `_EnableAlphaTest` is active, a `clip` call discards fragments below the alpha threshold before any displacement is applied, ensuring the hull respects hair card silhouettes. An `_OutlineDepthBias` value shifts clip-space Z toward the camera to prevent z-fighting at shallow angles.

The toon pass quantises the Lambertian dot product $\mathbf{N} \cdot \mathbf{L}$ per step using a per-band smoothstep. The expression after unification with the Avaturn version is:

```
float steps  = max(1.0, _ToonSteps);
float scaled = NdotL * steps;
float band   = floor(scaled);
float frac   = scaled - band;
float blend  = smoothstep(1.0 - _ToonSmoothness, 1.0, frac);
float toon   = saturate((band + blend) / steps);
toon = lerp(1.0 - _ShadowStrength, 1.0, toon);
```

Listing 5.1: V1 toon quantisation : Mixamo and Avaturn (unified).

Each band boundary receives an independent narrow blend of width `_ToonSmoothness`, so normal jitter near any seam produces a localised crossfade rather than snapping the whole surface patch between bands. Vertex positions and normals are read directly from the standard mesh attributes; no compute-skinning bridge is needed for standard `SkinnedMeshRenderer` avatars. An additive ambient `_AmbientColor` and a rim term `pow(1 - NdotV, RimPower)` are added to the quantised diffuse.

Avaturn

Shader: `Assets/Shaders/AvaturnNPRShaders/V1_ToonShading_GeometryOutline.shader`
Pipeline target: URP, `#pragma target 3.5`
Materials: six materials covering body, head, hair, eyelash, look, and shoe submeshes

The toon pass and property set are now identical to Mixamo. The earlier Avaturn shader carried a `_ToonThreshold` property — a vestige of a prototype that used a plain `step(threshold, NdotL)` comparison before the per-step smoothstep replaced it — but the value was declared and set in debug mode only and never read in the actual shading computation. It has been removed to align the two shaders. Both vertex passes include a conditional call to `OVR_FETCH_POS_NORM(v.vertex.xyz, v.normal, v.vertexID)`, which resolves to a no-op for standard GLTFast avatars. The bridge is carried through all subsequent technique versions for consistency.

All six materials share a common parameter set managed through the `AvaturnSlotPresets` editor: four toon bands (`_ToonSteps` = 4), a band-boundary softness of `_ToonSmoothness` = 0.05, a shadow fill factor of `_ShadowStrength` = 0.5, a world-space outline width of `_OuterOutlineWidth` = 0.005 m, and a rim-light exponent of `_RimPower` = 4. The sole per-submesh variation is alpha cutout: the hair and eyelash materials set `_EnableAlphaTest` = 1 with `_AlphaCutoff` = 0.1 and activate `_UseOutlineDepthOffset` to prevent z-fighting on the expanded hull over transparent card edges. The remaining four submeshes (body, head, look, shoe) leave alpha testing disabled, as their surfaces carry no transparency.

Meta Avatars SDK

Shader: `Assets/Shaders/CustomShaders/Avatar-Meta-UGB.shader`
Keyword: `EFFECT_TOON` (enabled via `ENABLE_NPR_EDGES`)
Technique file: `Assets/Shaders/CustomShaders/NPREffect_Toon.cginc`

The Meta implementation runs inside `AppSpecificPostManipulation`, which receives the fully composited PBR colour produced by the SDK's `STYLE_2_STANDARD` renderer. The quantisation therefore operates on RGB directly, preserving hue while stepping luminance:

```
float  bands      = max(2.0, _ToonColorBands);
float3 posterized = floor(color.rgb * bands + 0.5) / bands;
color.rgb = lerp(color.rgb, posterized, _ToonPosterizeStrength);

float lum  = dot(color.rgb, float3(0.2126, 0.7152, 0.0722));
```

```
color.rgb  = lerp(float3(lum, lum, lum), color.rgb, _ToonSaturation
    );
```

Listing 5.2: V1 posterisation : Meta SDK (`NPREffect_Toon.cginc`).

The `floor(rgb * bands + 0.5) / bands` expression rounds each channel to the nearest multiple of 1/bands (round-to-nearest rather than floor-to-band), which distributes quantisation error symmetrically and avoids the darkening bias of a plain floor. Saturation adjustment runs after posterisation to prevent clamping artefacts. Because the input already contains Meta's subsurface scattering, anisotropic hair, and eye glints, the posterised output stylises a physically correct image rather than reconstructing lighting from scratch.

The outline pass in this shader is a dedicated `NPROutline` pass with `Cull Front`, `ZWrite Off`, `ZTest LEqual`. The vertex shader displaces each SDK-skinned vertex outward by:

```
float3 worldPos = o.v_WorldPos + normalize(o.v_Normal) *
    _OutlineWidth * 0.001;
```

Listing 5.3: Outline vertex displacement : Meta SDK outline pass.

The `0.001` scale converts `_OutlineWidth` from millimetre-scale UI units to metre-scale world space. The outline is toggled via `_OutlineEnabled`; the fragment discards if the value is below 0.5. Unlike the Mixamo and Avaturn variants, `ZWrite Off` is used because the Meta pipeline already has correct depth from the main pass and a second depth write would cause artefacts with the SDK's transparency compositing.

Table 5.1: V1 implementation comparison across the three avatar platforms.

	Mixamo	**Avaturn**	**Meta SDK**
Quantisation method	Per-step smoothstep, NdotL	Per-step smoothstep, NdotL	Round-to-nearest, composited RGB
Skinning bridge	None	No-op macro (GLTFast SkinnedMeshRenderer)	SDK native
Rim light	Additive, computed	Additive, computed	Pre-composited by Meta PBR
Outline depth write	`ZWrite On`	`ZWrite On`	`ZWrite Off`
Outline toggleable	No	No	Yes (`_OutlineEnabled`)
Shadow source	None (`GetMainLight` without shadow coords)	None (same)	Pre-composited by Meta PBR

(a) Jody (Mixamo): four toon bands, NdotL per-step quantisation, additive rim and ambient.

(b) Avaturn: identical toon quantisation; hair and eyelash submeshes use alpha cutout (`_AlphaCutoff = 0.1`) and outline depth offset.

Figure 5.2: V1 (Toon Shading) on the Jody (Mixamo) and Avaturn platforms. Both avatars use four toon bands with per-step smoothstep blending. All submesh materials share the same shader parameters; the hair and eyelash slots differ only in enabling alpha cutout and outline depth offset to handle transparent card geometry. The Meta SDK implementation uses RGB posterisation on the composited PBR output and is presented separately above.

5.4.2 V1.2 — X-Toon Extended Toon Shading

Three shader implementations exist for V1.2, one per avatar platform. The outline pass is unchanged from V1 across all three; the forward-lit pass replaces the procedural quantisation of V1 with a 2D ramp texture lookup, adds an abstraction compression stage, and optionally applies Normal Field Abstraction [10] on the Mixamo and Avaturn platforms.

Mixamo

Shader: `Assets/AvatarShaderExperimental/Shaders/Shaders after Project/XToon_2DRamp.shader`
Pipeline target: URP, `#pragma target 3.5`

The forward-lit pass begins by optionally modifying the surface normal via Normal Field Abstraction. When `_UseAbstractNormals` is enabled, an *abstract normal map* — a simplified or blurred version of the surface normal map — is sampled, transformed from tangent space to world space through the TBN matrix, and linearly interpolated with the geometry normal:

```
float3 abstractN  = UnpackNormal(SAMPLE_TEXTURE2D(
                        _AbstractNormalMap,
    sampler_AbstractNormalMap, uv));
float3x3 TBN      = float3x3(tangentWS, bitangentWS, normalWS);
float3 abstractWS = normalize(mul(abstractN, TBN));
normalWS          = normalize(lerp(normalWS, abstractWS,
    _NormalSmoothing));
```

Listing 5.4: Normal Field Abstraction : Mixamo and Avaturn.

When `_NormalSmoothing` is high, surface normals within a smooth region converge toward each other, causing the 2D ramp to return a consistent colour band across that region and producing a graphic, flattened read of the surface shape.

The U coordinate for the ramp is derived from the Lambertian dot product modulated by URP shadow attenuation, then remapped and interpolated toward the ramp midpoint to control light sensitivity:

```
Light mainLight = GetMainLight(input.shadowCoord);
float3 lightDir = normalize(mainLight.direction);
float NdotL     = dot(normalWS, lightDir) * mainLight.
    shadowAttenuation;
float rampU     = lerp(0.5, saturate(NdotL * 0.5 + 0.5),
    _LightSensitivity);
```

Listing 5.5: U axis (lighting) computation : Mixamo and Avaturn.

The V coordinate (abstraction level) is computed from the fragment's world-space camera distance in depth mode, from the screen-space normal derivative magnitude in curvature mode, or from a constant in manual mode, selected at compile time via `shader_feature_local` keywords. The ramp is then sampled at `(rampU, rampV)`, and an abstraction compression stage flattens the lighting bands as the abstraction level rises:

```
float abstractU    = lerp(rampU, 0.5, rampV * 0.6);
float dynSmoothing = lerp(_RampSmoothing, _RampSmoothing + 0.35,
    rampV);
float shadowMask   = smoothstep(0.5 - dynSmoothing,
                                0.5 + dynSmoothing, abstractU);
float3 toonAlbedo    = albedo * rampColor;
float3 shadowedColor = lerp(toonAlbedo * _ShadowColor.rgb,
    toonAlbedo, shadowMask);
```

```
7 float3 finalColor    = lerp(albedo, shadowedColor, _ShadowStrength)
    ;
```

Listing 5.6: Abstraction compression and shadow mask : all platforms.

The expression $U_{\text{abs}} = \text{lerp}(U_{\text{raw}}, 0.5, V \cdot 0.6)$ pulls both lit and shadowed values toward the ramp midpoint as V increases, widening the perceived tonal range and producing a graphic fill at high abstraction levels. The smoothstep width `dynSmoothing` expands simultaneously, softening the band boundaries so that the transition between tones reads as drawn rather than mechanical. A Blinn-Phong specular term is added after the shadow stage:

```
1 float3 halfDir = normalize(lightDir + viewDir);
2 float NdotH    = dot(normalWS, halfDir);
3 float specular = smoothstep(1.0 - _SpecularSize -
    _SpecularSmoothness,
4                             1.0 - _SpecularSize +
    _SpecularSmoothness,
5                             NdotH) * mainLight.shadowAttenuation;
6 finalColor = lerp(finalColor, _SpecularColor.rgb, specular *
    _SpecularStrength);
```

Listing 5.7: Stylised specular (Blinn-Phong) : Mixamo and Avaturn.

The smoothstep width `_SpecularSmoothness` controls how hard or soft the edge of the highlight appears; setting it close to zero produces a sharp, ink-painted dot.

Avaturn

Shader: `Assets/Shaders/NPR/XToon_2DRamp.shader`
Pipeline target: URP, `#pragma target 3.5`
Materials: five materials covering body, head, hair, eyelash, and look submeshes

The Avaturn V2 shader shares the same forward-lit architecture as the Mixamo implementation: Normal Field Abstraction [10] via TBN, NdotL-based U axis, and the same abstraction compression and Blinn-Phong specular formulas. Both vertex passes include the SDK vertex fetch bridge, consistent with V1; for standard GLTFast meshes the bridge is a runtime no-op and adds no overhead.

The five per-submesh materials differ only in their texture assignments; all parameters — ramp texture, smoothing, abstraction mode, specular — are tuned uniformly across the set via the `AvaturnPresetShaderGUI` editor, which supports slot-based preset save and recall to maintain consistent looks across submeshes.

Meta Avatars SDK

Hook file: `Assets/Shaders/CustomShaders/NPREffect_XToon.cginc`
Keyword: `EFFECT_XTOON`
Entry point: `AppSpecificPostManipulation`

The Meta implementation runs inside `AppSpecificPostManipulation`. Unlike the Mixamo and Avaturn shaders, this hook does not own the vertex stage; world-space position is forwarded from the SDK vertex output as an explicit fifth argument to `ApplyNPREffect`. This makes the same U axis computation as the standalone platforms possible: `GetMainLight` is called with a per-fragment shadow coordinate derived from that world-space position, and the shadow-attenuated NdotL is computed identically to Listing 5.2:

```
float4 shadowCoord = TransformWorldToShadowCoord(positionWS);
Light mainLight    = GetMainLight(shadowCoord);
float3 lightDir    = normalize(mainLight.direction);
float NdotL        = dot(normalWS, lightDir) * mainLight.
    shadowAttenuation;
float rampU        = lerp(0.5, saturate(NdotL * 0.5 + 0.5),
    _XToonLightSensitivity);
```

Listing 5.8: U axis (shadow-attenuated NdotL) : Meta SDK (`NPREffect_XToon.cginc`).

The surface tangent frame is not available in the hook, so Normal Field Abstraction via an abstract normal map is not supported; only the geometric smoothing path is available on this platform. The abstraction compression and dynamic smoothing stages described in Listing 5.3 are applied identically. The V axis uses the same three-branch structure as Mixamo and Avaturn, selected at compile time via `multi_compile` keywords `_DETAILMODE_DEPTH`, `_DETAILMODE_CURVATURE`, and `_DETAILMODE_MANUAL`. Unlike the `shader_feature_local` dispatch on the standalone platforms, `multi_compile` generates all three variants as global shader variants, allowing the VR parameter panel to switch the active branch at runtime.

The same `lightDir` and shadow attenuation from the U axis computation are reused for the stylised specular term, matching the standalone implementations:

```
float3 halfDir  = normalize(lightDir + viewDir);
float NdotH     = dot(normalWS, halfDir);
float specular  = smoothstep(1.0 - _XToonSpecularSize -
    _XToonSpecularSmoothness,
                             1.0 - _XToonSpecularSize +
    _XToonSpecularSmoothness,
                             NdotH) * mainLight.shadowAttenuation;
finalColor = lerp(finalColor, _XToonSpecularColor.rgb, specular *
    _XToonSpecularStrength);
```

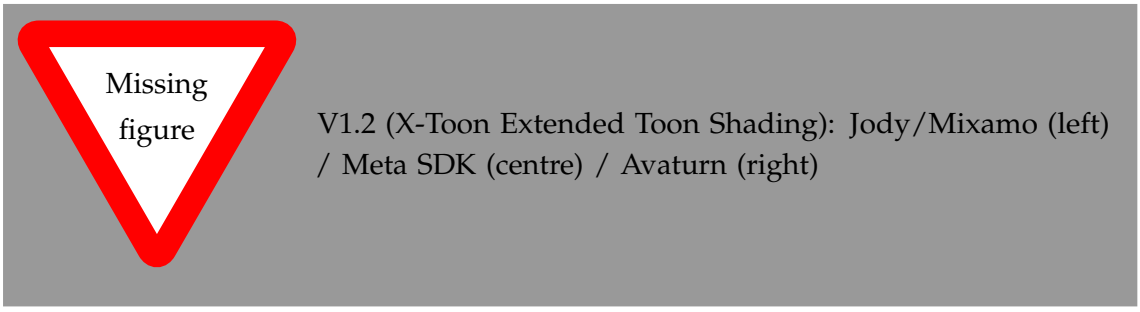
Listing 5.9: Stylised specular (shadow-attenuated) : Meta SDK.

All three V-axis modes — depth, curvature, and manual — were evaluated at the upper-body framing used in this study, with the avatar positioned approximately 1 m from the camera. At this distance, depth variation across the face and torso spans only a few centimetres; this range is too narrow relative to the configured depth window to produce spatial V-axis variation across pixels. Curvature mode was similarly unresponsive: the smooth, high-resolution normals of all three avatar platforms yield near-zero screen-space normal derivatives over most of the face, causing the V coordinate to remain at its minimum. As a result, no perceptible visual difference was observed between the three

modes. Depth mode was retained for the study figures, as it is the axis Barla et al. [10] use to motivate the technique. The images in Figure 5.3 were produced with the following parameters, applied uniformly across all three platforms: `_LightSensitivity` = 0.5, `_RampSmoothing` = 0.01, `_ShadowStrength` = 0.4, depth range 5–50 m, and an inverted-hull outline of width 0.005.

Table 5.2: V1.2 implementation comparison across the three avatar platforms.

	Mixamo	**Avaturn**	**Meta SDK**
U axis source	NdotL, shadow-attenuated	NdotL, shadow-attenuated	NdotL, shadow-attenuated (per-fragment `GetMainLight`)
V axis computation	`shader_feature_local` keywords (Depth / Curvature / Manual)	`shader_feature_local` keywords	`multi_compile` keywords (same three modes, runtime-switchable)
Normal Abstraction	TBN abstract normal map or geometric smoothing	TBN abstract normal map or geometric smoothing	Geometric smoothing only (no TBN in hook)
Specular model	Blinn-Phong, shadow-attenuated	Blinn-Phong, shadow-attenuated	Blinn-Phong, shadow-attenuated (reuses `GetMainLight` result)
Skinning bridge	None	None (GLTFast SkinnedMeshRenderer)	SDK native
Shadow source	Per-vertex `GetShadowCoord`, interpolated	Per-vertex `GetShadowCoord`, interpolated	Per-fragment `TransformWorldToShadowCoord`



V1.2 (X-Toon Extended Toon Shading): Jody/Mixamo (left) / Meta SDK (centre) / Avaturn (right)

Figure 5.3: V1.2 (X-Toon Extended Toon Shading) across the three avatar platforms.

5.4.3 V2 — Screen-Space Normal Edge Detection

Shaders: `Assets/Shaders/MetaAvatarShaders/NPREffect_NormalEdge.cginc` (Meta SDK); `Assets/Shaders/JadeNPRShaders/V2_NormalEdgeDetection.shader` (Mixamo); `Assets/Shaders/AvaturnNPR` (Avaturn)
Extra texture samples per fragment: 1 normal map sample (Mixamo / Avaturn, when `_UseNormalMap` enabled); 0 (Meta SDK, normal decode handled by SDK pipeline)

The technique applies the `ddx` and `ddy` hardware intrinsics to the world-space surface normal **N**, estimating how rapidly the normal changes across adjacent screen pixels. The gradient magnitude is:

$$M_N = \sqrt{\| \texttt{ddx}(\mathbf{N}) \|^2 + \| \texttt{ddy}(\mathbf{N}) \|^2} \tag{5.1}$$

A smoothstep converts M_N into an anti-aliased edge weight:

$$e_N = \texttt{smoothstep}(\tau - \sigma,\ \tau + \sigma,\ M_N) \cdot \alpha_N \tag{5.2}$$

where τ is `_NormalEdgeThreshold`, σ is `_NormalEdgeSmoothness`, and α_N is `_NormalEdgeStrength`.

Normal gradient edge (primary cue). On the Meta SDK platform, the world-space normal supplied to `ddx/ddy` is `i.geometry.normal` from `AppSpecificPostManipulation` — the SDK's fully composited fragment normal, which already incorporates the SDK's internal normal-map decode. No additional texture read is required:

```
float3 dNdx = ddx((float3)worldNormal);
float3 dNdy = ddy((float3)worldNormal);
float  M    = sqrt(dot(dNdx, dNdx) + dot(dNdy, dNdy));

float nMin = _NormalEdgeThreshold - _NormalEdgeSmoothness;
float nMax = _NormalEdgeThreshold + _NormalEdgeSmoothness;
float edge = smoothstep(nMin, nMax, M) * _NormalEdgeStrength;
```

Listing 5.10: V2 normal gradient and smoothstep edge : Meta SDK (`NPREffect_NormalEdge.cginc`).

On Mixamo and Avaturn, the normal must be decoded from the PBR normal map before the derivative is computed. The vertex shader builds the tangent frame from the mesh tangent attribute; the fragment shader then unpacks the normal map and transforms it from tangent space to world space via the TBN matrix:

```
// vertex shader output (Varyings)
float3 tangentWS   : TEXCOORD4;
float3 bitangentWS : TEXCOORD5;

// fragment shader -- decode normal map then apply derivatives
float3 geomNWS = normalize(IN.nWS);
float3 nWS     = geomNWS;

if (_UseNormalMap > 0.5)
{
    float3 normalTS = UnpackNormalScale(
        SAMPLE_TEXTURE2D(_BumpMap, sampler_BumpMap, IN.uv),
    _BumpScale);
    float3x3 TBN = float3x3(
        normalize(IN.tangentWS),
        normalize(IN.bitangentWS),
        geomNWS);
    nWS = normalize(mul(normalTS, TBN));
```

```
}

// nWS is then supplied to ddx/ddy as above
```

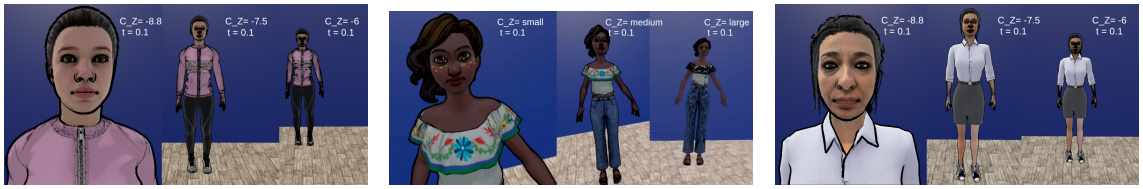
Listing 5.11: V2 normal map decode via TBN : Mixamo and Avaturn (`V2_NormalEdgeDetection.shader`).

When `_UseNormalMap` is disabled, `nWS` falls back to the bare interpolated geometry normal. The normal map files used are `Ch37_1001_Normal` and `Ch37_1002_Normal` for Jody, and the glTF-exported normal map from the Avaturn pipeline.

The distance-dependent artefact described in Section 4.3.4 arises directly from Equation 5.1: `ddx/ddy` are finite differences measured in screen-pixel units, so a fixed world-space crease projects onto fewer pixels as the avatar recedes, compressing the normal transition and increasing the apparent gradient magnitude. Edges grow thicker rather than thinner with distance, which motivated the UV-space albedo sampling approach of V3.

Table 5.3: V2 implementation comparison across the three avatar platforms.

	Jody (Mixamo)	**Avaturn**	**Meta SDK**
Normal source	TBN-decoded PBR normal map (baked)	TBN-decoded PBR normal map (glTF)	SDK composited fragment normal
Edge detail	Medium — normal map + mesh curvature	Dense — fine microdetail in normal map	Medium — SDK mesh anatomy
Sample cost	1 (normal map); 0 when disabled	1 (normal map); 0 when disabled	0 extra samples
Integration point	URP ForwardLit pass	URP ForwardLit pass	`AppSpecificPostManipulation`
Distance artifact	Edges thicken at range	Edges thicken at range	Edges thicken at range



(a) Jody (Mixamo): normal-gradient edges from TBN-decoded PBR normal map.

(b) Meta SDK: normal-gradient edges from SDK composited fragment normal.

(c) Avaturn: denser edge detail from high-resolution glTF normal map.

Figure 5.4: V2 (Screen-Space Normal Edge Detection) across all three avatar platforms. Edges are derived from screen-space normal gradients (`ddx/ddy`) and a Fresnel silhouette band. Avaturn produces finer interior detail due to its denser photogrammetry-derived normal map; the Meta SDK avatar uses the SDK composited normal directly with no additional texture sample.

5.4.4 V3 — Sobel Edge Detection

Shader: `Assets/Shaders/MetaAvatarShaders/NPREffect_Sobel.cginc` (Meta SDK); URP forward-lit pass with albedo sampling (Mixamo, Avaturn)
Extra texture samples per fragment: 9 (one per 3×3 neighbourhood position)

V3 moves from geometry-derived signals to the diffuse albedo texture as the edge source. Each of the nine 3×3 neighbourhood positions is sampled and converted to a scalar luminance using the ITU-R BT.601 luma coefficients [7]:

$$L = 0.299\,R + 0.587\,G + 0.114\,B \tag{5.3}$$

This prevents colour channel cross-talk from introducing false edges at hue boundaries where RGB channels change in opposite directions but luminance is constant. The Sobel kernels K_x and K_y are then applied to the nine luminance samples (kernels defined in Equation 3.2 in the background chapter).

```
float3 L      = float3(0.299, 0.587, 0.114);
float  offset = _SobelSampleDist * 0.001;

float tl = dot(tex2D(u_BaseColorSampler, uv+float2(-offset, offset)).rgb, L);
float t  = dot(tex2D(u_BaseColorSampler, uv+float2(      0, offset)).rgb, L);
float tr = dot(tex2D(u_BaseColorSampler, uv+float2( offset, offset)).rgb, L);
float l  = dot(tex2D(u_BaseColorSampler, uv+float2(-offset,      0)).rgb, L);
float r  = dot(tex2D(u_BaseColorSampler, uv+float2( offset,      0)).rgb, L);
float bl = dot(tex2D(u_BaseColorSampler, uv+float2(-offset,-offset)).rgb, L);
float b  = dot(tex2D(u_BaseColorSampler, uv+float2(      0,-offset)).rgb, L);
float br = dot(tex2D(u_BaseColorSampler, uv+float2( offset,-offset)).rgb, L);

float Gx      = (tr + 2*r + br) - (tl + 2*l + bl);
float Gy      = (tl + 2*t + tr) - (bl + 2*b + br);
float edgeMag = sqrt(Gx*Gx + Gy*Gy);
```

Listing 5.12: V3 nine-tap luminance sampling and Sobel gradient (`NPREffect_Sobel.cginc`).

UV seam boundaries produce spurious detections: texels on either side of a seam carry distant UV coordinates, so the colour difference registers as a strong edge even though no visual boundary exists. The suppression computes the luminance spread across all nine samples and rejects detections where the spread exceeds `_SobelSeamLimit`. A second upper bound `_SobelMax` clips spike magnitudes that exceed any plausible structural edge:

```
float lMin = min(min(min(tl,t),min(tr,l)), min(min(r,bl),min(b,br)));
float lMax = max(max(max(tl,t),max(tr,l)), max(max(r,bl),max(b,br)));

float seamMask = step(lMax - lMin, _SobelSeamLimit);   // reject seam spikes
```

```
float inBand   = step(_SobelThreshold, edgeMag)
               * step(edgeMag, _SobelMax);            // accept only valid range

float edge = inBand * seamMask * _SobelStrength;
color.rgb  = lerp(color.rgb, _InnerLineColor.rgb, edge);
```

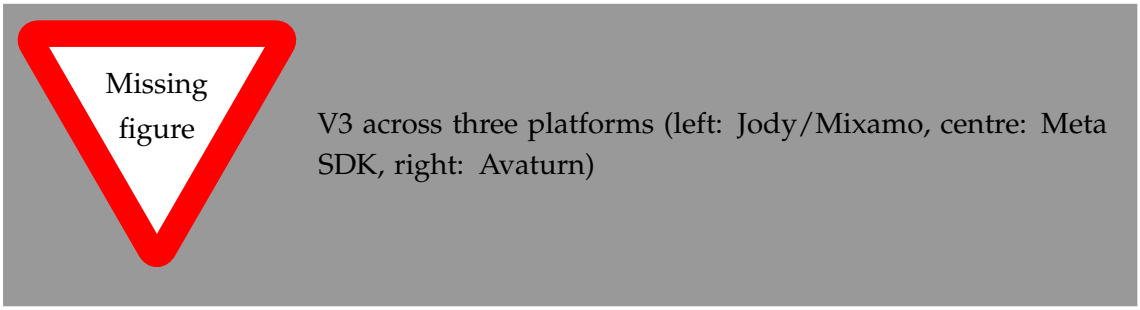
Listing 5.13: Seam suppression and magnitude band gate.

The UV-space offset `_SobelSampleDist` is distance-independent, which resolves the distance artifact of V2: the sampling neighbourhood stays fixed in texture space regardless of how far the avatar is from the camera.

On Mixamo and Avaturn, `u_BaseColorSampler` reads the raw diffuse albedo, so edge detection responds to surface material boundaries. On the Meta SDK, the hook receives only the composited PBR output; the Sobel therefore responds to shadow boundaries and specular gradients in addition to material transitions, which reduces selectivity but still adds visible clothing-level edges and broad facial contour lines.

Table 5.4: V3 implementation comparison across the three avatar platforms.

	Jody (Mixamo)	**Avaturn**	**Meta SDK**
Sobel input	Raw diffuse albedo texture	Raw diffuse albedo texture	Composited PBR colour
Luminance weights	BT.601 (0.299 / 0.587 / 0.114)	BT.601	BT.601
Sample count	9 albedo samples	9 albedo samples	9 samples on PBR output
Seam suppression	Luma range gate + upper magnitude bound	same	same
Distance artifact	None (UV-space offset)	None	None
Shadow edges	No (pre-lighting albedo)	No	Yes (PBR composite)



V3 across three platforms (left: Jody/Mixamo, centre: Meta SDK, right: Avaturn)

Figure 5.5: V3 (Sobel Edge Detection with seam suppression) across the three avatar platforms.

5.4.5 V3.2 — Gaussian Prefiltered Sobel

Shaders: `Assets/Shaders/MetaAvatarShaders/NPREffect_GaussianSobel.cginc` (Meta SDK); `Assets/Shaders/JadeNPRShaders/V4_GaussianPreFilteredSobel.shader` (Mixamo); `Assets/Shaders/AvaturnNPRShaders/V4_GaussianPreFilteredSobel.shader` (Avaturn)
Keyword (Meta): `EFFECT_GAUSS_SOBEL`
Extra texture samples per fragment: up to 72 (8 Sobel positions × 9 Gaussian taps) when blur is enabled; 8 when disabled

The pre-filter is implemented as a 9-tap weighted luminance helper (`GaussianLuma` on the Meta SDK, `SampleLuminanceBlurred` on Mixamo and Avaturn). At each call, one centre sample, four axis-aligned cardinal samples, and four diagonal corner samples are drawn at a configurable offset from the given UV position. The three weight groups are normalised to unity before use so that the output is always a valid luminance estimate:

```
float GaussianLuma(float2 center, float blurR, float cW, float
    cardW, float diagW)
{
    float3 L = float3(0.299, 0.587, 0.114);
    float  v = 0.0;
    v += dot(tex2D(u_BaseColorSampler, center).rgb, L) * cW;
    v += dot(tex2D(u_BaseColorSampler, center + float2( blurR,
    0)).rgb, L) * cardW;
    v += dot(tex2D(u_BaseColorSampler, center + float2(-blurR,
    0)).rgb, L) * cardW;
    v += dot(tex2D(u_BaseColorSampler, center + float2(     0,
    blurR)).rgb, L) * cardW;
    v += dot(tex2D(u_BaseColorSampler, center + float2(     0, -
    blurR)).rgb, L) * cardW;
    v += dot(tex2D(u_BaseColorSampler, center + float2( blurR,
    blurR)).rgb, L) * diagW;
    v += dot(tex2D(u_BaseColorSampler, center + float2(-blurR,
    blurR)).rgb, L) * diagW;
    v += dot(tex2D(u_BaseColorSampler, center + float2( blurR, -
    blurR)).rgb, L) * diagW;
    v += dot(tex2D(u_BaseColorSampler, center + float2(-blurR, -
    blurR)).rgb, L) * diagW;
    return v;
}

// Weight normalisation: computed once before the nine Sobel
    positions are sampled
float totalW = _GSobelCenterWeight + 4.0*_GSobelCardinalWeight +
    4.0*_GSobelDiagonalWeight;
totalW = max(totalW, 0.0001);
cW    = _GSobelCenterWeight   / totalW;
cardW = _GSobelCardinalWeight / totalW;
diagW = _GSobelDiagonalWeight / totalW;
```

Listing 5.14: 9-tap Gaussian luminance helper with weight normalisation (`NPREffect_GaussianSobel.cginc`).

When `_GSobelEnableGaussBlur = 0` the weights collapse to centre = 1.0, cardinal = 0, diagonal = 0, making the helper a plain point sample that reproduces V3 behaviour at no additional cost. The helper is called once at each of the nine Sobel neighbourhood positions, and the G_x / G_y kernels are applied to the nine resulting luminance values using the same formula as V3. After computing the gradient magnitude, a configurable threshold band gates the result: `smoothstep(_GSobelThreshold * _GSobelThreshMin, _GSobelThreshold * _GSobelThreshMax, edgeMag)`. The upper ceiling of this band suppresses extreme-magnitude detections such as UV seam spikes, replacing the explicit luma-range seam gate of V3.

A series of four progressive smoothstep passes then sharpens the thresholded magnitude into a crisp edge mask:

```
float hw1 = lerp(0.5, 0.03, _GSobelTightness);
edge = smoothstep(0.5 - hw1, 0.5 + hw1, edge);
float hw2 = lerp(0.5, 0.15, _GSobelTightness);
edge = smoothstep(0.5 - hw2, 0.5 + hw2, edge);
float hw3 = lerp(0.5, 0.25, _GSobelTightness);
edge = smoothstep(0.5 - hw3, 0.5 + hw3, edge);
float hw4 = lerp(0.5, 0.35, _GSobelTightness);
edge = smoothstep(0.5 - hw4, 0.5 + hw4, edge);
edge = pow(edge, _GSobelPowerCurve);
edge *= _GSobelStrength;
```

Listing 5.15: 4-pass progressive smoothstep sharpening and power curve (`NPREffect_GaussianSobel.cginc`).

At `_GSobelTightness = 0` every half-width equals 0.5, reducing each pass to a linear identity; at `_GSobelTightness = 1` the widths shrink to 0.03, 0.15, 0.25, and 0.35, composing a steep sigmoid. `_GSobelPowerCurve > 1` suppresses soft halos; values below 1 amplify thin lines.

Platform differences. On the Meta SDK, all four smoothstep passes are always evaluated and driven by a single `_GSobelTightness` scalar. The Mixamo and Avaturn shaders (`V4_GaussianPreFilteredSobel.shader`) expose the four passes as individually toggleable flags (`_EnablePass1`–`_EnablePass4`) and offer three preset configurations for rapid cross-avatar setup: Light filtering uses the classical kernel weights (centre 0.25, cardinal 0.125, diagonal 0.0625, blur multiplier 0.8, power curve 1.5); Moderate uses a wider footprint (blur multiplier 1.0, centre 0.3, power curve 2.0); Aggressive applies a narrow threshold band (0.85–1.15), maximum tightness, and a steep power curve of 3.0. The default parameter set on all three platforms corresponds to the Light configuration.

Table 5.5: V3.2 implementation comparison across the three avatar platforms.

	Jody (Mixamo)	**Avaturn**	**Meta SDK**
Gaussian input	Raw diffuse albedo	Raw diffuse albedo (glTF)	Composited PBR output
Default kernel weights	centre 0.25 / cardinal 0.125 / diagonal 0.0625	same	same
Seam suppression	Threshold band (min/max multipliers)	same	same
Smoothstep passes	4, individually toggleable (`_EnablePass1`–`_EnablePass4`)	same	4, always on via `_GSobelTightness`
Preset configurations	Light / Moderate / Aggressive	same	N/A
Sample count (blur on)	Up to 72	Up to 72	Up to 72

Figure 5.6: V3.2 (Gaussian Prefiltered Sobel) across the three avatar platforms.

5.4.6 V4 — Halftone Screening and Cross-Hatching

Standalone shader: `Assets/Shaders/CommonNPRShaders/HalftoneHatching.shader`
Meta SDK hooks: `NPREffect_Halftone.cginc`, `NPREffect_Hatching.cginc`
Keywords: `_PATTERNMODE_HALFTONE`, `_PATTERNMODE_HATCHING`, `_PATTERNMODE_STIPPLE`, `_PATTERNMODE_COMBINED` (standalone); `EFFECT_HALFTONE`, `EFFECT_HATCHING` (Meta SDK)

The standalone shader used by both Mixamo and Avaturn implements four selectable pattern modes via `shader_feature_local` keywords. The tone value is computed once per fragment from the scene lighting before being supplied to the active pattern function:

```
Light mainLight = GetMainLight(input.shadowCoord);
float NdotL     = saturate(dot(normalWS, mainLight.direction));
float tone      = saturate(NdotL * mainLight.shadowAttenuation +
    _ToneBias);
```

Listing 5.16: Tone derivation from shadow-attenuated NdotL: standalone (Mixamo and Avaturn).

Halftone Pattern

The halftone function generates a regular dot grid. UV coordinates are first rotated by `_HalftoneAngle` degrees, then tiled at frequency `_HalftoneScale`. The cell-local position after recentring determines the radial distance from the dot centre; a smoothstep edge softens the dot boundary:

```
float2 rotCoords = Rotate2D(coords, _HalftoneAngle);
float2 gridPos   = frac(rotCoords * _HalftoneScale) - 0.5;
float  dist      = length(gridPos);
float  dotRadius = sqrt(max(0.0, 1.0 - tone)) * 0.5;
float  sharpInv  = 0.5 / max(_HalftoneSharpness, 0.001);
float  pattern   = 1.0 - smoothstep(dotRadius - sharpInv,
                                    dotRadius + sharpInv, dist);
```

Listing 5.17: Halftone dot grid: standalone and Meta SDK (identical logic).

The `sqrt` maps the linear tone value to dot area, matching the photomechanical relationship between darkness and ink coverage. The `max(0.0,...)` guard prevents a domain error for tone values marginally above one.

Hatching Pattern (TAM Approximation)

The hatching function implements four progressive line layers. Each layer is a single-direction line pattern computed by rotating UV coordinates and sampling the fractional X position:

```
float t = 1.0 - tone;   // darkness (0 = white, 1 = black)

// Layer 1: primary direction
if (t > 0.15)
    pattern = max(pattern, HatchLine(coords, _HatchAngle,
    _HatchThickness)
                          * smoothstep(0.15, 0.40, t));

// Layer 2: cross direction
if (t > 0.35)
    pattern = max(pattern, HatchLine(coords, _CrossHatchAngle,
    _HatchThickness)
                          * smoothstep(0.35, 0.60, t));

// Layer 3: dense diagonal
if (t > 0.55)
    pattern = max(pattern, HatchLine(coords,
                               (_HatchAngle + _CrossHatchAngle) *
    0.5,
                               _HatchThickness * 1.5)
                          * smoothstep(0.55, 0.80, t));

// Layer 4: near-black fill
if (t > 0.80)
```

```
22     pattern = max(pattern, smoothstep(0.80, 1.0, t));
```

Listing 5.18: TAM-approximation hatching: four-layer progressive activation.

Each layer uses `max` accumulation so that any activated layer can only add marks, never remove those from a lighter layer, reproducing the nesting property of a TAM [12]. The `smoothstep` on each layer's darkness range blends the layer in progressively rather than switching it on with a hard step, preventing temporal popping during animation.

Ink and Paper Colour Model

Both patterns share the same compositing stage. A paper colour and an ink colour are each optionally tinted with the surface albedo via `_TextureInfluence`; the mark pattern then selects between the two:

```
float3 paperCol   = lerp(_PaperColor.rgb, baseAlbedo,
   _TextureInfluence);
float3 inkCol     = lerp(_InkColor.rgb, baseAlbedo * _InkColor.rgb,
    _TextureInfluence);
float3 markColor  = lerp(paperCol, inkCol, pattern);
color.rgb         = lerp(color.rgb, markColor, _Strength);
```

Listing 5.19: Ink/paper compositing: standalone and Meta SDK (identical).

Meta SDK

The Meta implementation splits halftone and hatching into two separate include files, each exporting `ApplyNPREffect`. Both files replicate the pattern functions above with identical code. The only structural difference is the tone source: the hook receives the composited PBR colour and derives tone from its luminance:

```
float lum  = dot(color.rgb, float3(0.299, 0.587, 0.114));
float tone = saturate(lum + _HTToneBias);   // halftone
// float tone = saturate(lum + _HatToneBias); // hatching
```

Listing 5.20: Tone derivation from PBR luminance: Meta SDK (`NPREffect_Halftone.cginc`, `NPREffect_Hatching.cginc`).

The composited luminance already encodes shadow, ambient, and subsurface contributions from the SDK renderer. Mark density therefore responds to the full physically accurate lighting without requiring a separate NdotL computation inside the hook.

Table 5.6: V4 implementation comparison across the three avatar platforms.

	Jody (Mixamo)	**Avaturn**	**Meta SDK**
Shader file	`HalftoneHatching.shader`	`HalftoneHatching.shader`	`NPREffect_Halftone.cginc` / `NPREffect_Hatching.cginc`
Tone source	NdotL, shadow-attenuated	NdotL, shadow-attenuated	BT.601 luminance of composited PBR
Pattern functions	Halftone, Hatching, Stipple, Combined	same	Halftone or Hatching (separate files)
Coordinate space	UV / Screen / World (keyword)	UV / Screen / World (keyword)	UV only
Normal map decode	`rgb * 2.0 - 1.0` (GLTFast)	`rgb * 2.0 - 1.0` (GLTFast)	SDK composited normal (no decode)
OVR skinning bridge	None	None	SDK native

The normal map decode uses the raw-RGB GLTFast convention for both standalone platforms because the shared `HalftoneHatching.shader` was authored against Avaturn's GLTFast pipeline. On Jody, whose FBX-imported normal maps use Unity's DXT5nm format, this produces a slight colour shift if the normal map feature is enabled; disabling `_NORMALMAP` on Jody's halftone materials eliminates the artefact. The OVR skinning bridge present in the V1 through V3.2 Avaturn shaders is absent from the shared common shader; for standard GLTFast `SkinnedMeshRenderer` avatars this has no runtime effect.

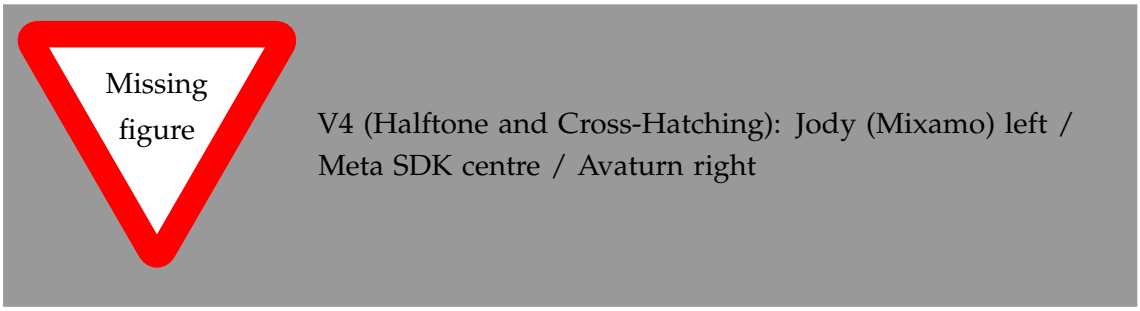


Figure 5.7: V4 (Halftone Screening and Cross-Hatching) across the three avatar platforms.

5.4.7 V5 — Hierarchical Edge Detection

The hierarchical implementation computes three independent edge layers in a single fragment shader invocation and combines them via weighted addition.

The depth layer uses `ddx` and `ddy` applied to the fragment's linear eye depth to estimate the depth gradient magnitude. The gradient is normalised by `_DepthSensitivity` and thresholded to produce a binary edge signal. The normal layer applies the same derivative approach to the interpolated world-space surface normal vector, computing $M_N = \sqrt{\|\texttt{ddx}(\mathbf{N})\|^2 + \|\texttt{ddy}(\mathbf{N})\|^2}$ and thresholding against `_NormalThreshold`. The colour layer applies the Roberts Cross operator to the diffuse texture: it samples four neighbours in a 2×2 diagonal pattern and computes the cross-difference magnitudes $|R_1|$ and $|R_2|$ as defined in Section 3.2.1, thresholded against `_ColorThreshold`.

The three binary edge signals are weighted by `_DepthWeight`, `_NormalWeight`, and `_ColorWeight` respectively and summed. The combined weight is clamped to $[0, 1]$ and used to blend between the base albedo and `_EdgeColor`.

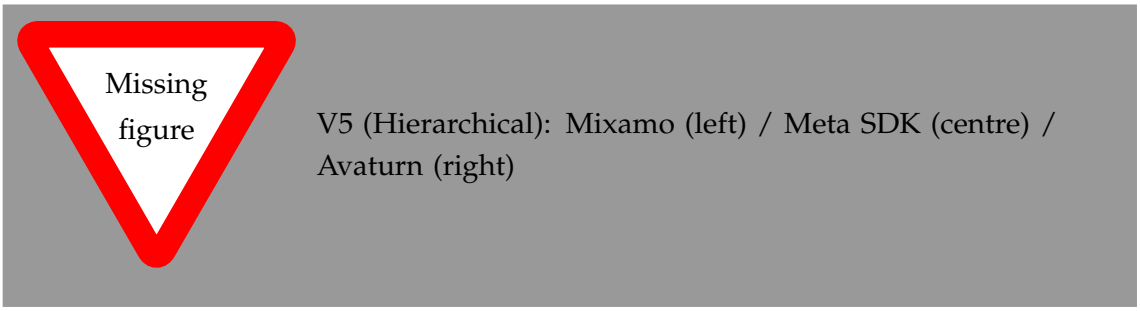


Figure 5.8: V5 (Hierarchical Edge Detection) across the three avatar platforms.

5.4.8 V6 — Kuwahara Painterly Filter

The Kuwahara implementation evaluates four overlapping quadrant subregions around the current fragment. Each quadrant is a square of radius `_KuwaharaRadius`, offset from the centre so that adjacent quadrants share a one-texel border. For each quadrant, the implementation accumulates the sum and sum-of-squares of the sampled colour values, then computes the per-channel mean and variance from these accumulators. The quadrant with the lowest luminance variance is selected, and its mean colour is written as the fragment output.

Because the filter operates on the diffuse texture using fixed texture-coordinate offsets, no intermediate render target is required, and the entire computation runs inside the single-pass fragment hook. The isotropic formulation is used; the anisotropic extension of Kyprianidis et al. [11] requires a structure tensor computed in a prior pass and is therefore incompatible with the single-pass constraint of the Meta SDK hook.

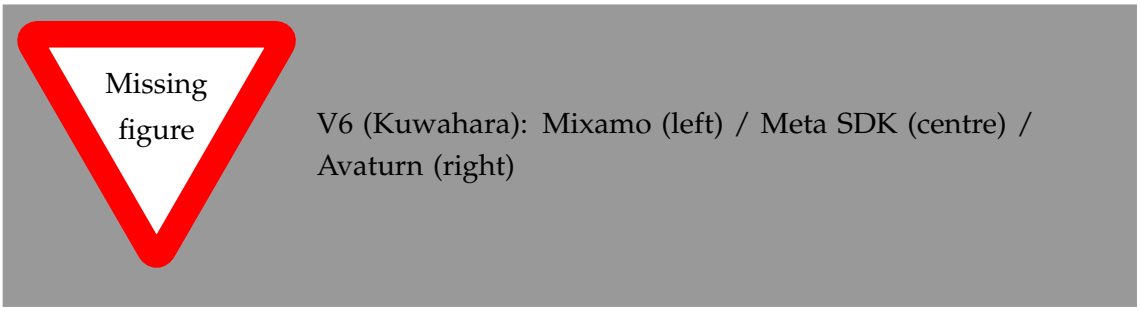


Figure 5.9: V6 (Kuwahara Painterly Filter) across the three avatar platforms.

5.4.9 V7 — Kuwahara with Sobel Edges

The V7 implementation chains the Kuwahara and Sobel stages in sequence within the same fragment shader invocation. The Kuwahara filter is applied first, producing a smoothed mean colour from the lowest-variance quadrant. The Sobel gradient is then computed using the Kuwahara output as the input signal rather than the raw diffuse

texture. Because the Kuwahara output is piecewise uniform within regions, the Sobel operator responds primarily to inter-region boundaries, suppressing the fine texture gradients that produce noise in V3. The skin saturation and seam rejection checks from V3 are retained. The detected edges are composited over the Kuwahara output using `_EdgeColor` and `_EdgeThreshold`.

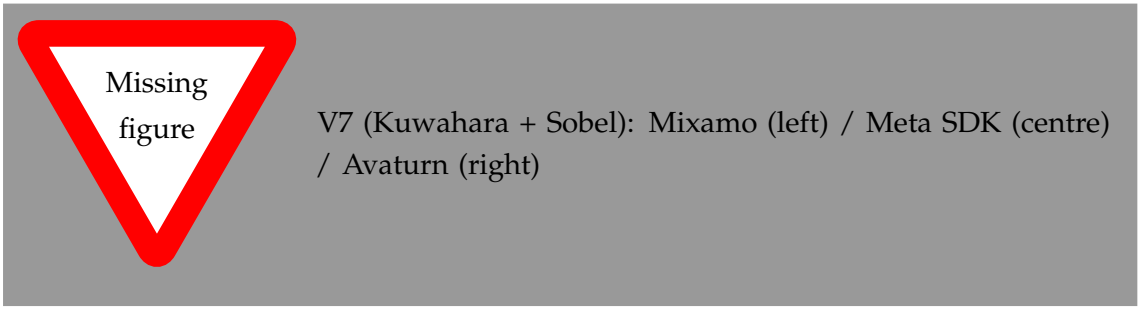


Figure 5.10: V7 (Kuwahara with Sobel Edges) across the three avatar platforms.

5.4.10 V8 — Composite: Kuwahara, Gaussian Sobel, and Hierarchical

The composite technique chains three stages. First, the Kuwahara filter produces the painterly base. Second, the Gaussian prefiltered Sobel is applied to the Kuwahara output to extract smoothed colour boundary lines. Third, the hierarchical depth, normal, and Roberts Cross edge layers are evaluated and composited on top of the Gaussian Sobel result with independent weighting. All three stages share the same input texture and are computed sequentially within the single fragment shader invocation. The total sample count is the sum of the Kuwahara quadrant samples, the Gaussian prefilter samples, and the four Roberts Cross samples, making this the most sample-intensive technique developed to date.

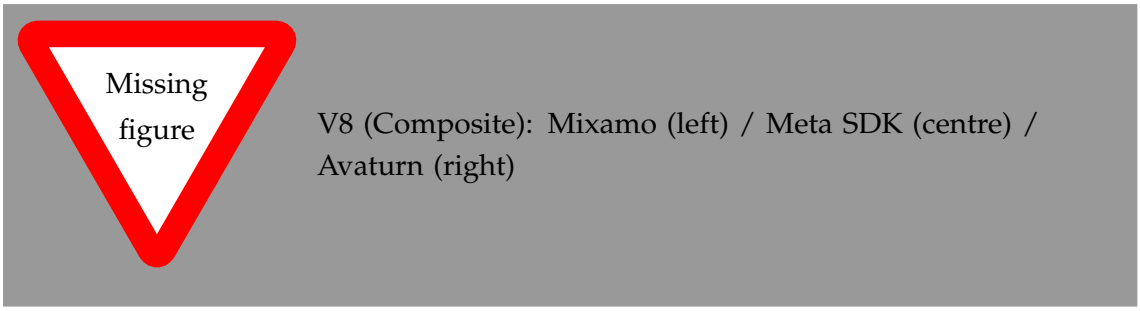


Figure 5.11: V8 (Composite: Kuwahara, Gaussian Sobel, Hierarchical) across the three avatar platforms.

5.5 Runtime Tooling

5.5.1 In-VR Parameter Tuning Interface

Shader parameters can be adjusted through two routes: the Unity Inspector during offline development, and a custom in-VR WorldSpace panel during live headset sessions.

The Inspector provides the standard Unity material property interface, allowing each parameter to be set numerically or through drag controls on the material asset before building. The WorldSpace panel, implemented in `NPREdgeDetectionUI.cs`, provides the same controls in the running application so that parameter values can be explored and compared while the avatar is visible in the headset at correct VR scale.

The panel is constructed at runtime from Unity UI primitives: each tunable parameter is represented by a labelled row containing either a slider or a toggle button, laid out in technique-specific groups. Controller raycasting using the OVR interaction layer detects when the ray from either controller intersects a slider's `BoxCollider`; a trigger-pull gesture initiates a drag interaction that maps horizontal ray movement to the slider's value range. Technique switching is handled through a dropdown row: selecting a technique calls `EnableKeyword` and `DisableKeyword` on the active materials to swap the active shader branch without recompilation. The panel exposes parameters across all technique groups, including per-technique thresholds, weights, kernel radii, and colour properties, and was central to the visual assessment process used to select the technique for the user study.

5.5.2 LOD Material Management

`AvatarShaderSwapper.cs` solves the material reversion problem described in Section 5.2.2. On startup it subscribes to the avatar load completion callback and performs an initial material replacement pass across all LOD levels. A coroutine running at a configurable polling interval inspects the active materials on each renderer at runtime; if an SDK-assigned material is detected — indicating that a LOD transition has reverted the custom assignment — the coroutine re-applies the NPR material and copies the base colour and normal texture references from the newly active SDK material into the replacement shader's properties.

5.5.3 Pose Freeze for Evaluation

`AvatarFreezeController.cs` enables the avatar's body-tracking pose to be frozen at a chosen instant, preserving a specific expression and body posture for evaluation screenshots. The freeze is implemented by accessing the `_inputTrackingProvider` field of `OvrAvatarInputManager` via reflection and substituting a snapshot provider that returns a fixed pose state. Pressing A, X, or the Space key toggles the freeze on and off.

6 Evaluation

This chapter evaluates the implemented NPR avatar system and documents the planned user study for trust assessment.

6.1 Technical Evaluation Setup

6.1.1 Evaluation Objectives

The technical evaluation verifies:

- the quality and coherence of NPR avatar rendering in VR
- real-time performance and stability under realistic usage
- readiness of the system for a trust-focused user study.

6.1.2 Technical Metrics

Define the quantitative and qualitative metrics:

- average frame rate and minimum frame rate
- frame time variance and rendering latency
- stylization fidelity indicators such as edge coherence and temporal stability
- subjective observations from developer-led test sessions.

6.1.3 Baseline Comparisons

List the technical baselines used for comparison:

- non-stylized VR avatar rendering
- existing NPR avatar rendering approaches, if available
- simplified style variants to isolate the effect of each component.

6.2 Implementation Validation

6.2.1 Performance Results

Summarize measured performance:

- frame rate and latency results in representative VR scenarios
- resource usage on target hardware
- performance behavior for each style variant.

6.2.2 Visual Quality Assessment

Report qualitative observations and comparisons:

- visual consistency across head motion
- edge quality, shading coherence, and stylization clarity
- artifacts or failure cases observed during testing.

6.2.3 Stability and Robustness

Describe the stability of the implementation:

- behavior during extended VR sessions
- integration issues and debugging outcomes
- support for varying avatar animations and scene complexity.

6.3 User Study Design

This section describes the planned user study that will assess how NPR avatar styles influence trust.

6.3.1 Study Goals

The study aims to answer:

- how do different NPR avatar styles affect perceived trustworthiness in VR?
- which visual traits most strongly influence trust and social presence?
- does performance or visual quality moderate trust judgments?

6.3.2 Participants and Recruitment

Define the participant pool and recruitment plan:

- target sample size and demographics
- inclusion and exclusion criteria
- ethical considerations and informed consent.

6.3.3 Experimental Conditions

Describe the conditions participants will experience:

- baseline avatar rendering
- multiple NPR avatar variants with different stylization levels
- within-subject or between-subject study design.

6.3.4 Task Scenario

Outline the interaction scenario used in the study:

- task or dialog with the avatar
- session duration and condition order
- behavioural measures such as task completion or response time.

6.3.5 Measurement Instruments

List the trust and presence instruments:

- trust questionnaire adapted from VR or HRI research
- presence / immersion scale
- post-condition ratings for comfort, realism, and trust
- optional behavioural metrics such as interaction choices.

6.3.6 Pilot Study Plan

Describe the pilot study to validate the protocol:

- a small pilot participant set
- testing condition order, questionnaire clarity, and timing
- refining instructions and VR content before the full study.

6.4 Discussion

Reflect on the implementation and readiness for the user study.

6.4.1 Readiness for the User Study

Summarize whether the current system is ready for the planned study and what remains to be done.

6.4.2 Limitations

Discuss limitations affecting the planned study:

- hardware and platform constraints
- asset and style coverage limitations
- potential bias in the selected stylization conditions.

7 Conclusion

7.1 Summary of Contributions

This thesis developed and evaluated a system for applying non-photorealistic rendering to real-time VR avatars across three distinct avatar platforms, with the aim of understanding both the technical feasibility of NPR integration and the perceptual consequences of visual abstraction for avatar trust. The work was structured around three research questions: whether NPR can be integrated into platforms with different rendering architectures, which technique achieves the best trade-off between visual quality and performance, and how visual abstraction affects the perceived trustworthiness of a speaking avatar.

The first contribution is an extensible multi-platform NPR rendering system implemented in Unity 6 with URP, supporting three avatar platforms — Mixamo, Avaturn, and the Meta Avatars SDK — that differ substantially in their shader accessibility and runtime management. Eight stylisation techniques were designed and implemented across this system, each motivated by a specific limitation of its predecessor. The sequence progresses from a geometry-based inverted hull outline through screen-space derivative edge detection, Sobel and Gaussian Sobel colour-boundary detection, hierarchical multi-cue edge fusion, and the isotropic Kuwahara painterly filter, ending in two composite techniques that chain multiple abstraction strategies within a single fragment shader invocation. For the Meta SDK platform, all effects were implemented inside the `AppSpecificPostManipulation` hook of the Extensible Shader Framework, which is the only integration point that does not break the SDK's skinning, LOD management, or PBR material system.

The second contribution is the adaptation of the X-Toon extended toon shading technique [10] to a production avatar context, including a Normal Field Abstraction mechanism on the Mixamo and Avaturn platforms that uses a simplified normal map and TBN-space interpolation to progressively flatten surface shape toward its low-frequency form, and an abstraction compression formula that widens shading bands as the detail axis increases. On the Meta platform, where intermediate lighting terms are not accessible within the post-processing hook, a luminance-based proxy replaces the direct Lambertian dot product, and the light direction is sourced from the URP per-frame global `_MainLightPosition.xyz`, enabling a correct Blinn-Phong specular response.

The third contribution is a runtime tooling infrastructure supporting iterative visual assessment in VR. The fully procedural in-VR parameter panel (`NPREdgeDetectionUI.cs`) exposes over sixty tunable parameters across all technique groups via controller-driven sliders and supports real-time technique switching without shader recompilation. A preset save and recall system allows parameter configurations to be stored to persistent storage and reloaded across sessions, supporting systematic exploration of the parameter space. A material management script (`AvatarShaderSwapper.cs`) resolves the Meta SDK's LOD-driven material reversion problem through a polling coroutine, ensuring that custom NPR assignments persist through all SDK LOD transitions.

7.2 Limitations

Several limitations of the current system are worth noting. The Meta SDK integration is constrained to the `AppSpecificPostManipulation` hook, which fires after the PBR lighting stage; techniques that require access to intermediate lighting terms, per-fragment NdotL, or the tangent frame are therefore not directly realisable on the Meta platform. This limitation required separate implementations for the X-Toon and toon shading techniques, introducing a divergence in the parameter space across platforms that complicates direct visual comparison.

The Kuwahara filter implementation uses the isotropic formulation, which produces rotationally symmetric smoothing kernels. The anisotropic extension of Kyprianidis et al. [11], which aligns the smoothing kernel with local image structure, would produce stronger feature preservation at region boundaries but requires a structure tensor computed in a preceding render pass. Because additional render passes are not feasible within the single-pass Meta SDK hook, and because consistency across platforms was a design goal, the anisotropic formulation was excluded.

Performance on the Meta Quest target hardware was not formally benchmarked in this thesis. The sample counts for the higher-complexity techniques — Gaussian Sobel, the composite Kuwahara pipeline — scale with kernel radius, and the worst-case fragment cost was not profiled across the full range of avatar mesh complexity and viewing distances. This leaves open the question of whether the most sample-intensive techniques run within the Quest's frame budget under the conditions of the user study.

The user study evaluates a single NPR condition (the technique selected through visual assessment) against a photorealistic baseline and a real-person baseline, using a fixed avatar identity and a fixed set of recommendation scenarios. The design does not vary the degree of abstraction within the NPR condition, so it cannot measure how trust responds to the specific strength of stylisation applied. The choice of Avaturn as the study platform means the study measures the effect of NPR applied to a photorealistic AI-reconstructed avatar specifically; responses may differ for more stylised avatar types such as Mixamo.

7.3 Future Work

Several directions extend naturally from this work. The most direct extension is a parametric study of abstraction degree: rather than comparing a single NPR condition to a photorealistic baseline, future work could vary the `_LightingStrength` or `_XToonLightingStrength` blend parameter continuously to measure how trust, uncanny valley ratings, and social presence co-vary with abstraction level. This would produce a more complete mapping of the trust-abstraction curve than the single-point comparison of this thesis.

On the technical side, the Meta SDK hook constraint could potentially be relaxed in future SDK versions that expose earlier pipeline stages; access to per-fragment NdotL and the tangent frame would enable Normal Field Abstraction and direct NdotL toon shading on the Meta platform, eliminating the current cross-platform divergence. Alternatively,

a deferred-injection approach using Unity's Render Graph could apply NPR effects to the Meta avatar in a separate pass outside the SDK hook, at the cost of a second depth sample.

The temporal stability of screen-space edge detection under head motion in VR is a known challenge that was addressed during development through threshold calibration but not through explicit temporal filtering. A moving-average filter on the edge magnitude signal, or a reprojection-based approach that warps edge maps between frames, could reduce the flickering that screen-space derivatives produce at seam boundaries when the avatar is in motion.

Finally, the avatar identity used in the user study was designed to be neutral and gender-unspecified; future work could investigate whether the trust effect of NPR stylisation interacts with avatar identity characteristics such as perceived gender, age, or species, extending the study design toward the broader question of how the form of an avatar mediates social responses to its behaviour.

Bibliography

[1] K. L. Nowak and F. Biocca, "The effect of the agency and anthropomorphism on users' sense of telepresence, copresence, and social presence in virtual environments," *Presence: Teleoperators and Virtual Environments*, vol. 12, no. 5, pp. 481–494, 2003. DOI: 10.1162/105474603322761289.

[2] F. Weidner and W. Broll, "Watching yourself in a video conference: The role of perspective and avatar embodiment," in *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, 2023. DOI: 10.1145/3544548.3581551.

[3] M. Mori, "The uncanny valley," *Energy*, vol. 7, no. 4, pp. 33–35, 1970, Translated by Karl F. MacDorman and Norri Kageki.

[4] A. Gooch, B. Gooch, P. Shirley, and E. Cohen, "A non-photorealistic lighting model for automatic technical illustration," in *Proceedings of SIGGRAPH*, 1998, pp. 447–452. DOI: 10.1145/280814.280950.

[5] A. Lake, C. Marshall, M. Harris, and M. Blackstein, "Stylized rendering techniques for scalable real-time 3D animation," in *Proceedings of the 1st International Symposium on Non-Photorealistic Animation and Rendering (NPAR)*, 2000, pp. 13–20. DOI: 10.1145/340916.340918.

[6] T. Isenberg, B. Freudenberg, N. Halper, S. Schlechtweg, and T. Strothotte, "A developer's guide to silhouette algorithms for polygonal models," *IEEE Computer Graphics and Applications*, vol. 23, no. 4, pp. 28–37, 2003. DOI: 10.1109/MCG.2003.1210862.

[7] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th. Pearson, 2018, ISBN: 9780133356724.

[8] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986. DOI: 10.1109/TPAMI.1986.4767851.

[9] D. Marr and E. Hildreth, "Theory of edge detection," *Proceedings of the Royal Society of London. Series B, Biological Sciences*, vol. 207, no. 1167, pp. 187–217, 1980. DOI: 10.1098/rspb.1980.0020.

[10] P. Barla, J. Thollot, and L. Markosian, "X-Toon: An extended toon shader," in *Proceedings of the 4th International Symposium on Non-Photorealistic Animation and Rendering (NPAR)*, 2006, pp. 127–132. DOI: 10.1145/1124728.1124749.

[11] J. E. Kyprianidis, H. Kang, and J. Döllner, "Image and video abstraction by anisotropic Kuwahara filtering," *Computer Graphics Forum*, vol. 28, no. 7, pp. 1955–1963, 2009. DOI: 10.1111/j.1467-8659.2009.01574.x.

[12] E. Praun, H. Hoppe, M. Webb, and A. Finkelstein, "Real-time hatching," in *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 2001, pp. 581–586. DOI: 10.1145/383259.383328.

[13] R. Canales, D. Roble, and M. Neff, "The impact of avatar stylization on trust," in *2024 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*, 2024, pp. 418–428. DOI: 10.1109/VR58804.2024.00063.

[14] K. F. MacDorman and H. Ishiguro, "The uncanny advantage of using androids in cognitive and social science research," in *Proceedings of the ICDL-2006 Workshop on Android Science*, 2006, pp. 1–9.

[15] M. E. Latoschik, D. Roth, D. Gall, J. Achenbach, T. Waltemate, and M. Botsch, "The effect of avatar realism in immersive social virtual realities," in *Proceedings of the 23rd ACM Symposium on Virtual Reality Software and Technology (VRST)*, ACM, 2017. DOI: 10.1145/3139131.3139156.

[16] N. Yee and J. N. Bailenson, "The Proteus effect: The effect of transformed self-representation on behavior," *Human Communication Research*, vol. 33, no. 3, pp. 271–290, 2007. DOI: 10.1111/j.1468-2958.2007.00299.x.

[17] Adobe Inc., *Mixamo: 3d character rigging and animation service*, Online service for rigged humanoid character export, 2024.

[18] Meta Platforms, Inc., *Meta avatars SDK documentation*, https://developer.oculus.com/documentation/unity/meta-avatars-overview/, Accessed: 2025-01-10, 2024.

[19] A. Atteneder, *glTFast: A runtime glTF loader for Unity*, Open-source package for runtime glTF 2.0 loading in Unity, 2023.

[20] C. Schlick, "An inexpensive BRDF model for physically-based rendering," *Computer Graphics Forum*, vol. 13, no. 3, pp. 233–246, 1994. DOI: 10.1111/1467-8659.1330233.

[21] J. F. Blinn, "Models of light reflection for computer synthesized pictures," in *Proceedings of the 4th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1977, pp. 192–198. DOI: 10.1145/965141.563893.

Additional Results

This appendix contains supplementary results that support the findings in Chapter 6 but were omitted from the main body for conciseness.

1 Extended Quantitative Comparison

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2 Additional Qualitative Examples

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

3 Hyperparameter Sensitivity

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.